API By FYERS

FYERS APIS

Fyers API is a set of **REST-like APIs** that provide integration with our `in-house` trading platform with which you can build your own customized trading applications. To view the old version of docs, click **here**

Introduction

Fyers API is a set of **REST-like APIs** that provide integration with our in-house trading platform with which you can build your own customized trading applications. You can place fresh single or multiple orders, modify and cancel existing orders in real-time. You can also get account related information such as orderbook, tradebook, net positions, holdings, and funds.

We have ensured maximum security for our APIs which prevent unauthorised transactions. All API requests and received only over HTTPS protocol.

You can read more about when we introduced FYERS APIs **here**

Libraries and SDKs

Certainly! To make it even easier for you to use the Fyers API in different programming languages, we have provided dedicated libraries/packages that handle the API calls for you. These libraries/packages abstract away the complexities of raw HTTP calls, allowing you to focus on integrating the API seamlessly into your applications. Below are the links to the libraries/packages.

- **Fyers Python library** - Supports Python 3.8 to 3.12 version
- **Fyers Node.js library** - Supports Node.js 12 to 21.6.2 version
- **Fyers Web JS library** - Supports in Browser
- **Fyers C# library** - Supports .NET 8.0.4
- **Fyers Java library** - Supports Java 8

CDN Link

Versions	Links
1.2.1	https://cdn.fyers.in/js/sdk/1.2.1/fyers-web-sdk-v3/index.min.js
1.2.0	https://cdn.fyers.in/js/sdk/1.2.0/fyers-web-sdk-v3/index.min.js
1.1.0	https://cdn.fyers.in/js/sdk/1.1.0/fyers-web-sdk-v3/index.min.js
1.0.0	https://cdn.fyers.in/js/sdk/1.0.0/fyers-web-sdk-v3/index.min.js

Community

We have a dedicated community to discuss, share and raise feature requests on FYERS API. Our goal is to empower the AlgoTrading community in India with the most robust and easy to integrate APIs.

You can interact with us on our dedicated topic on **FYERS API** on **FYERS Community**

Request & Response Structure

Everything about Request & Response Structure

Authorization Headers

Once the authentication is completed, you will receive an access_token. For all of the following requests, you will be required to send a combination of appId and access_token (api_id:access_token) in the HTTP Authorization header.

Request samples

cURL

Copy

```
curl -H "Authorization: api_id:access_token"
curl -H "Authorization: aaa-99:bbb"
```

Success

Response Attributes

Attribute	Data Type	Description
`s`	string	“ok”
`code`	int	200
`message`	string	“”
`Additional key`	object / list / int / string	Each request will contain its own key based on the request

Failure

The error response attributes will contain the following

Response Attribute

Attribute	Data Type	Description
`s`	string	“error”
`code`	int	Negative integer to identify the specific error
`message`	string	Error message to identify error
`HTTP Header`	int	Refer to the error codes table View Details

HTTP Status Codes

The status codes contain the following

Status Code	Meaning
200	Request was successful
400	Bad request. The request is invalid or certain other errors
401	Authorization error. User could not be authenticated
403	Permission error. User does not have the necessary permissions
429	Rate limit exceeded. Users have been blocked for exceeding the rate limit.
500	Internal server error.

Common API Error Codes

The status codes contain the following

Code	Description
-8	This error occurs when token is Expired
-15	This error occurs when Invalid token is provided
-16	This error occurs when server is unable to authenticate user token
-17	This error occurs when token is passed is either Invalid or Expired
-50	This error occurs when one or more invalid parameters are passed. The message field in the API response will include details about the specific invalid inputs.
-51	This error occurs when invalid Order ID is passed while fetching Orders or Order Modification

-53	This error occurs when invalid position ID is passed
-99	This error occurs when order placement get rejected with rejection message in the API response
-300	This error occurs when invalid symbol provided
-352	This error occurs when invalid App ID is provided
-352	This error occurs in exit position API, if no position available to exit
-429	This error occurs when the API rate limit is exceeded, either per second, minute, or day.
400	This error occurs in multi leg order placement when invalid input passed

Rate Limit

Timeframe	Rate Limit
Per Second	10
Per Minute	200
Per Day	100000

Permission Templates

You can provide different app permissions for each application at the time of creation.

Permission Template	Basic	Transactions Info	Order Placement	Market Data
			1. Transactions	

List of activities allowed	1. Profile Details 2. Logout App 3. Logout	1. Basic Included 2. Orders 3. Positions 4. Trades 5. Holdings 6. Funds 7. Market Status	Info Included 2. Order Placement 3. Order Modification 4. Order Cancellation 5. Exit Positions 6. Convert Positions	1. Historical data 2. Market Depth 3. Quotes

User blocking

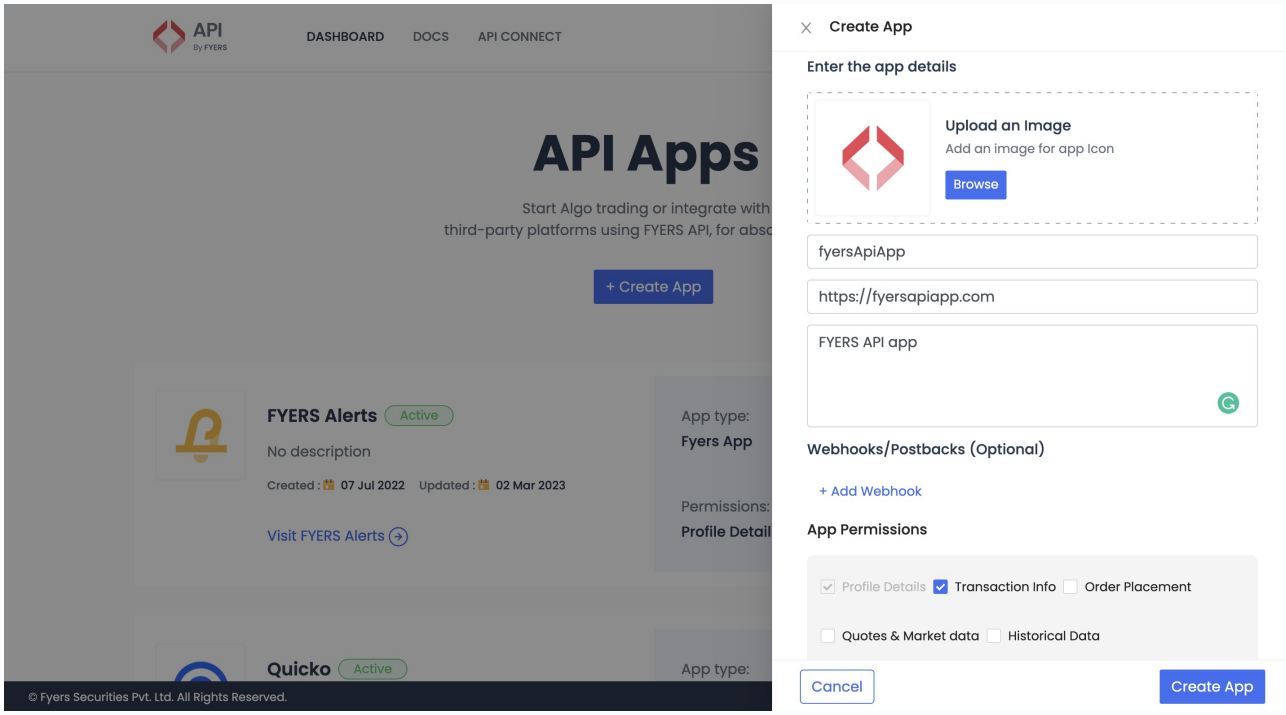
The user will be blocked for the rest of the day if the per minute rate limit is exceeded more than 3 times in the day.

App Creation

Individual Apps

These are apps which are created for your own personal usage. These apps can be used only by the creator of the app and no other client can login and make use of this particular app.
To create an app, you need to follow the following steps:-

1. Login to API **Dashboard**
2. Click on Create App
3. Provide the following details
    - App Icon
    - App Name
    - Redirect URL
    - Description (Optional)
    - App Permissions - Refer Permissions Template



Third Party Apps

These apps are used by platform providers which would allow end users to login to the app and make use of the functionality. These apps are created by third party application providers to enable FYERS clients to use their applications.
To create a common app, you can get in touch with us at **api-support@fyers.in**.

Redirect URI

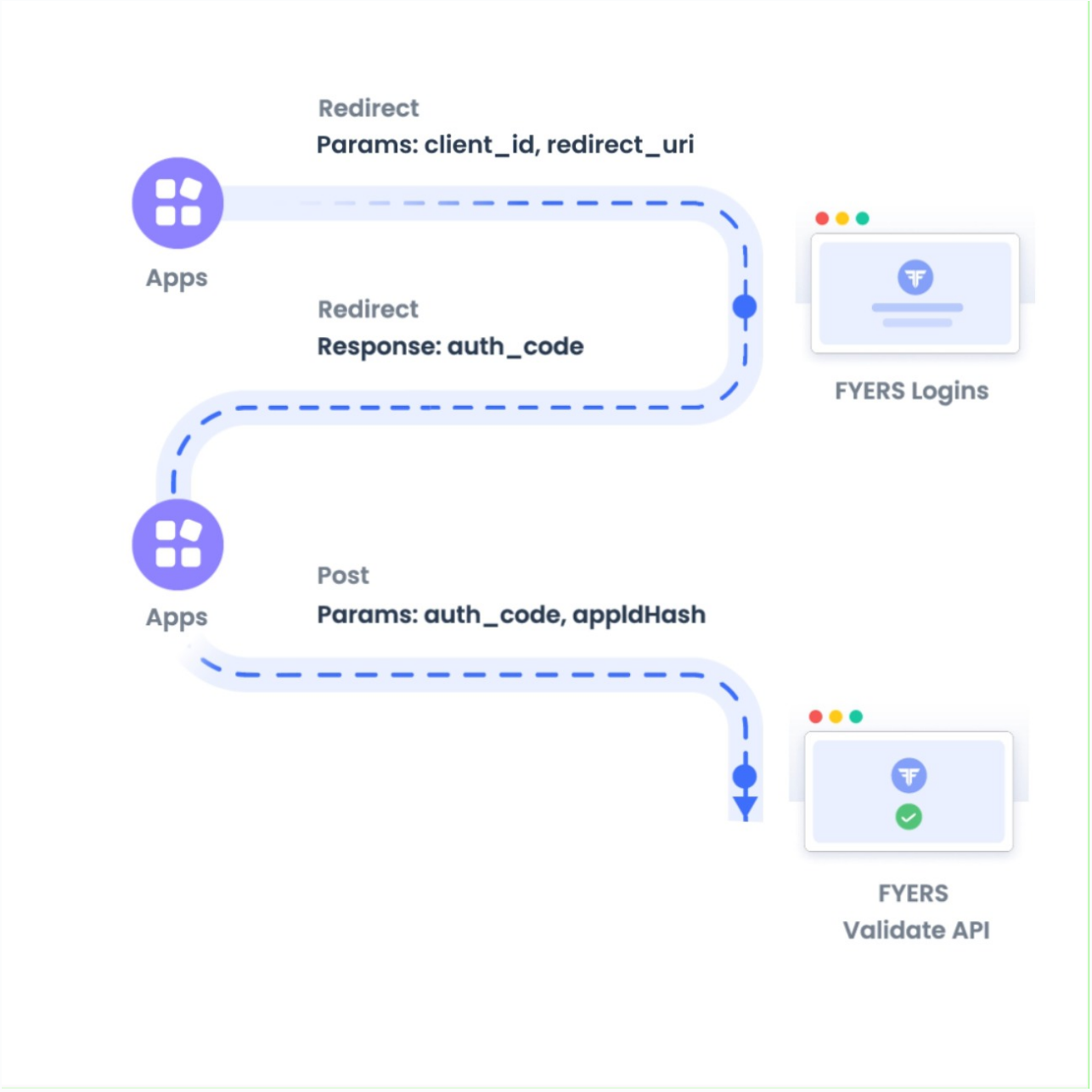
The user will be redirected to the redirect uri after successfully logging in using the FYERS credentials. The redirect uri should be in your control as the auth token is sensitive information.

Authentication & Login Flow - User Apps

Authentication Steps

The login flow is as follows:

1. Navigate to the Login API endpoint
2. After successful login, user is redirected to the redirect uri with the auth_code
3. POST the auth_code and appIdHash (SHA-256 of api_id + app_secret) to Validate Authcode API endpoint
4. Obtain the access_token use that for all the subsequent requests



Request Parameters for Step 1

Request Attributes

Attribute	Data Type	Description
`client_id`	string	This is the app_id which you have received after creating the app. Eg: “qwerty-100”
`redirect_uri`	string	This is where the user will be redirected after successful login. Eg: https://trade.fyers.in/api-login/redirect-uri/index.html **This should be the same as what was provided at the time of app creation**
`response_type`	string	This value must always be “code”
`state`	string	You send a random value. The same value will be returned after successful login to the redirect uri. Eg: “abcdefg”

Response Attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	This is the code to identify specific responses
`message`	string	This is the message to identify the specific error responses
`auth_code`	string	String value which will be used to generate the access_token
`state`	string	This value is returned as is from the first request

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
# Import the required module from the fyers_apiv3 package
from fyers_apiv3 import fyersModel

# Replace these values with your actual API credentials
client_id = "SPXXXXE7-100"
secret_key = "N********B"
redirect_uri = "https://trade.fyers.in/api-login/redirect-uri/index.html"
response_type = "code"
state = "sample_state"

# Create a session model with the provided credentials
session = fyersModel.SessionModel(
    client_id=client_id,
    secret_key=secret_key,
    redirect_uri=redirect_uri,
    response_type=response_type
)

# Generate the auth code using the session model
response = session.generate_authcode()

# Print the auth code received in the response
print(response)
```

```
--------------------------------------------------------------------------------
SAMPLE SUCCESS RESPONSE :
--------------------------------------------------------------------------------

https://api-t1.fyers.in/api/v3/generate-authcode?
client_id=SPXXXXE7-100&
redirect_uri=https%3A%2F%2Fdev.fyers.in%2Fredirection%2Findex.html
&response_type=code&state=sample_state&nonce=sample_nonce
```

Request Parameters for Step 2

Request Attributes

Attribute	Data Type	Description
`grant_type`	string	This value must always be “authorization_code
`appIdHash`	string	SHA-256 of api_id + app_secret Eg: SHA-256 of app_id:app_secret is 7c7120d2b5004f8de22d8dc2da0453b4d7e6211e37a4108b8371266ecff00498 You can use this online tool for reference
`code`	string	This is the auth_code which is received from the first step

Response Attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	This is the code to identify specific responses
`message`	string	This is the message to identify the specific error responses
`access_token`	string	This value will be used for all the subsequent requests.

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
# Import the required module from the fyers_apiv3 package
from fyers_apiv3 import fyersModel

# Define your Fyers API credentials
client_id = "SPXXXXE7-100"  # Replace with your client ID
secret_key = "N********B"  # Replace with your secret key
redirect_uri = "https://trade.fyers.in/api-login/redirect-uri/index.html"  #
response_type = "code"
grant_type = "authorization_code"

# The authorization code received from Fyers after the user grants access
auth_code = "eyJ0eXAi*******.eyJpc3MiOiJhcGkubG9********.r_65Awa1kGdsNTAgD***

# Create a session object to handle the Fyers API authentication and token ge
```

```
session = fyersModel.SessionModel(
    client_id=client_id,
    secret_key=secret_key,
    redirect_uri=redirect_uri,
    response_type=response_type,
    grant_type=grant_type
)

# Set the authorization code in the session object
session.set_token(auth_code)

# Generate the access token using the authorization code
response = session.generate_token()

# Print the response, which should contain the access token and other details
print(response)
```

```
--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------

{
  's': 'ok',
  'code': 200,
  'message': '',
  'access_token': 'eyJ0eXAiOi***.eyJpc3MiOiJh***.HrSubihiFKXOpUOj_7***',
  'refresh_token': 'eyJ0eXAiO***.eyJpc3MiOiJh***.67mXADDLrrleuEH_EE***'
}
```

Refresh Token

When we validate the auth code to generate the access token, a refresh token is also sent in the response.

The refresh token has a validity of 15 days. A new access token can be generated using the refresh token as long as the refresh token is valid.

The following parameters must be passed in the body for the POST request

Attribute	Data Type	Description
`grant_type`	string	The grant_type parameter must be set to “refresh_token”.
`appIdHash`	string	SHA-256 of api_id + app_secret. Eg: SHA-256 of app_id:app_secret c7120d2b5004f8de22d8dc2da0453b4d7e6211e37a4108b8371266ec You can use this online tool for reference (https://emn178.github.io/o tools/sha256.html)
`refresh_token`	string	The refresh token previously issued to the client from the validate au API
`pin`	string	The user's pin

Request samples

cURL

Copy

```
curl --location --request POST 'https://api-t1.fyers.in/api/v3/validate-refre
--header 'Content-Type: application/json' \
--data-raw '{
  "grant_type": "refresh_token",
  "appIdHash": "c3efb1075ef2332b3a4ec7d44b0f05c1********************",
  "refresh_token": "eyJ0eXAiOiJKV1***.eyJpc3MiOiJhcGkuZn***.5_Qpnd1nQXBw1T_wN
  "pin": "****"
}'

--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------

{
    "s": "ok",
    "code": 200,
    "message": "",
    "access_token": "eyJ0eXAiOiJK***.eyJpc3MiOiJhcGkuZnllcnM***.IzcuRxg4tnXiU
}
```

Best Practices

These are the recommended best practises that you should follow while using the APIs

1. Never share your app_secret with anyone
2. Never share your access_token with anyone
3. Do not provide trading permissions unless you want to use the app to place orders
4. Provide a redirect_uri which is in your control rather than a public endpoint such as google.com
5. You should send a random value in the state parameter and verify whether the same value has been returned to you

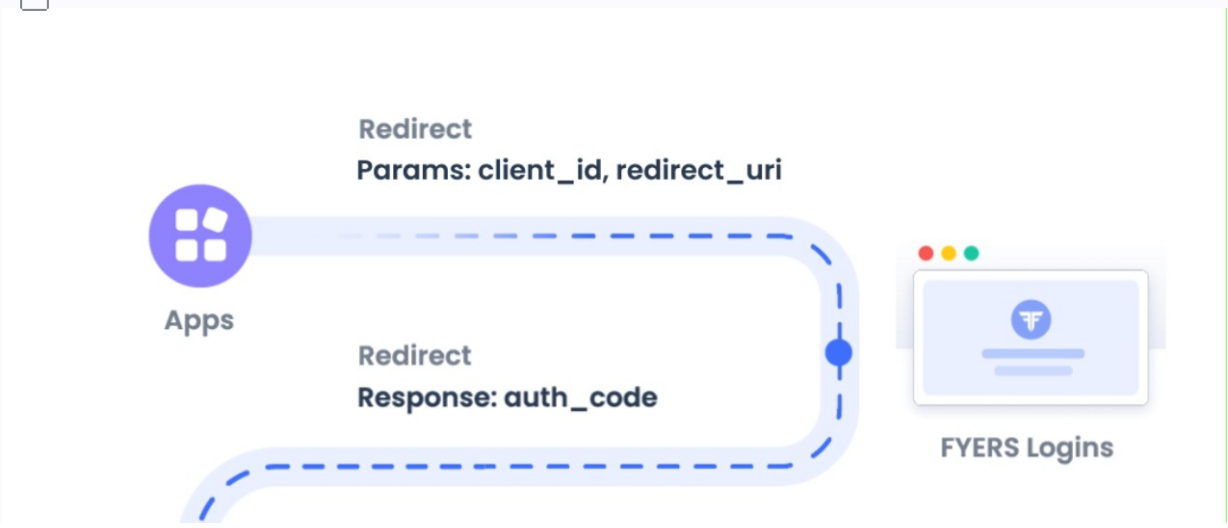
Authentication & Login Flow - Third Party Apps

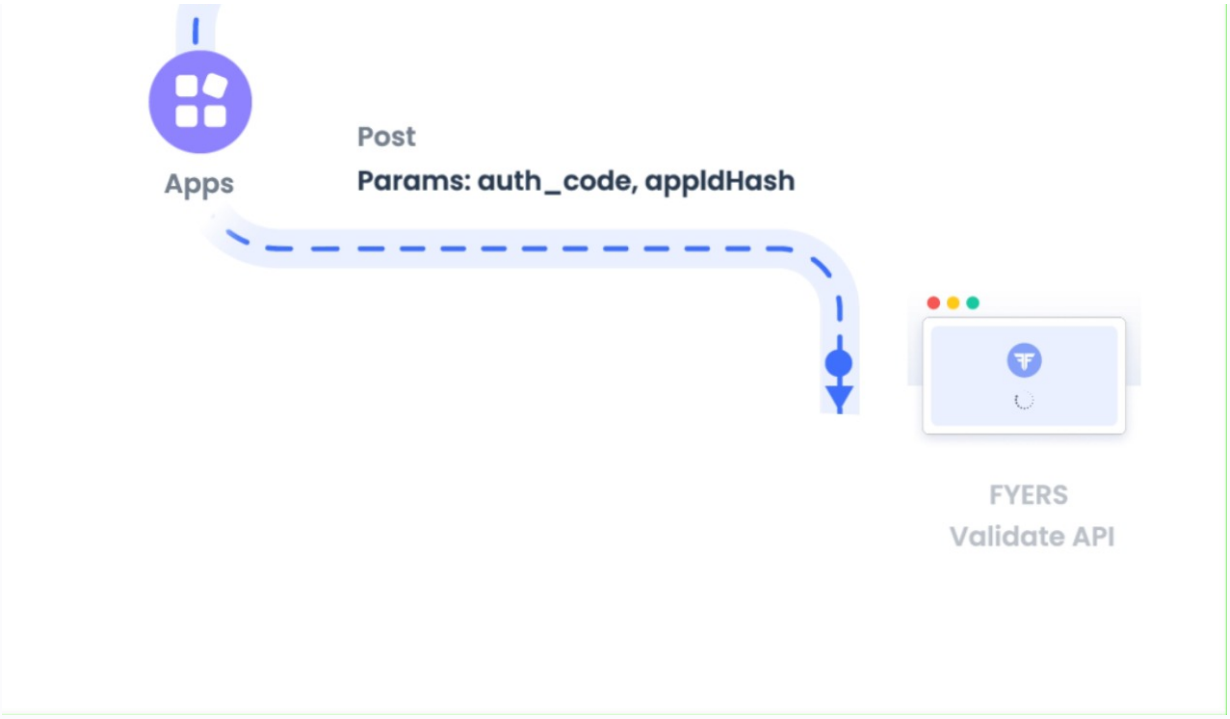
OAuth2 is authentication flow for Third Party Apps

OAuth2 - Auth Flow

This is a simple OAuth 2 Authentication Flows.

- This is recommended for applications which have a backend server which can authenticate the second step
- This is **not recommended** for Single Page Applications (SPA)
- **Diagram**





- **Request Parameters for Step 1**

  You need to navigate the user to the FYERS login url with the correct get parameters

Request Attributes

Attribute	Data Type	Description
`client_id`	string	This is the app_id which you have received after creating the app. Eg: “qwerty-102”
`redirect_uri`	string	This is where the user will be redirected after successful login. Eg: https://trade.fyers.in/api-login/redirect-uri/index.html **This should be the same as what was provided at the time of app creation**
`response_type`	string	This value must always be “code”
`state`	string	You send a random value. The same value will be returned after successful login to the redirect uri. Eg: “abcdefg”

Response Attributes

Attribute	Data Type	Description
`s`	string	ok / error

Attribute	Data Type	Description
`code`	int	This is the code to identify specific responses
`message`	string	This is the message to identify the specific error responses
`auth_code`	string	String value which will be used to generate the access_token
`state`	string	This value is returned as is from the first request

- **Request Parameters for Step 2**

Request Attributes

Attribute	Data Type	Description
`grant_type`	string	This value must always be “authorization_code”
`appIdHash`	string	SHA-256 of api_id + app_secret Eg: SHA-256 of app_id:app_secret is c7120d2b5004f8de22d8dc2da0453b4d7e6211e37a4108b8371266e You can use this online tool for reference
`code`	string	This is the auth_code which is received from the first step

Response Attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	This is the code to identify specific responses
`message`	string	This is the message to identify the specific error responses
`access_token`	string	This value will be used for all the subsequent requests.

Request samples

cURL

Copy

```
curl -H "Content-Type: application/json" -X POST -d
   '{
         "grant_type":"authorization_code","appIdHash":"d8f152a3c455add2bd636
   }' https://api-t1.fyers.in/api/v3/validate-authcode

 ------------------------------------------------------------------------------
 Sample Success Response
 ------------------------------------------------------------------------------

 {
   "s": "ok",
   "code": 200,
   "message": "",
   "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJhcGkuZnll
 }
```

Best Practices

These are the recommended best practises that you should follow while using the APIs

- Never share your app_secret with anyone
- Never share your access_token with anyone
- Do not provide trading permissions unless you want to use the app to place orders
- Provide a redirect_uri which is in your control rather than a public endpoint such as google.com
- You should send a random value in the state parameter and verify whether the same value has been returned to you
- Do not store the app_secret in the front end. It should be securely kept without exposing it to third parties.

Sample Code

Postman Collection

We have created an extensive Postman collection which will make it easier for implementation. Kindly import the postman_collection & postman_environment_variables

- Download and import the postman collection from **here**
- Download and import the postman environment variables from **here**
- You can check the sample Script to get started with from **here**
- We have provided dummy data in the environment variables. Kindly update the correct values after you import it.

User

FyersModel Class (Python SDK)

When initialized, it requires parameters such as client_id and access_token for authentication.This class supports synchronous execution by default (is_async=False). Additionally, it enables logging functionality, allowing users to specify a log path (log_path) where logs will be stored. By default, the logging level is set to 'error', but users have the option to set it to 'debug' for more detailed logging information.

Arguments in FyersModel Class

Arguments	Data Type	Description
`client_id`	string	For authentication
`access_token`	string	For authentication
`isAsync(optional)`	boolean	Default value is False for synchronous flow, for asynchronous set it True
`log_path(optional)`	string	Specify the file path where you want to store the logs
`log_level(optional)`	string	By default, the logging level is set to 'ERROR', level can be set 'DEBUG' for more detailed logs.

Profile

This allows you to fetch basic details of the client.

Response Attributes

Attribute	Data Type	Description
`name`	string	Name of the client
`display_name`	string	Display name, if any, provided by the client
`fy_id`	string	The client id of the fyers user
`image`	string	URL link to the user's profile picture, if any.
`email_id`	string	Email address of the client
`pan`	string	PAN of the client
`pin_change_date`	string	Date when last PIN was updated
`pwd_change_date`	string	Date when last password was updated
`mobile_number`	string	Registered mobile number
`totp`	boolean	Status of TOTP
`pwd_to_expire`	int	Number of days until the current password expires
`ddpi_enabled`	boolean	Status of DDPI
`mtf_enabled`	boolean	Status of MTF

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, is_async=False, token=acce

# Make a request to get the user profile information
response = fyers.get_profile()

# Print the response received from the Fyers API
print(response)
```

```
--------------------------------------------------------------------------------
Sample Response
--------------------------------------------------------------------------------
```


```
  {
    "s":"ok",
    "code":200,
    "message":"",
    "data":{
        "fy_id":"XXXXX86",
        "name":"VINAY",
        "image":"https://fyers-user-details.s3.amazonaws.com/image/FK61075482
        "display_name":"VKM",
        "pin_change_date":"15-12-2022 09:24:05",
        "email_id":"xxxnayxm@gmxxl.com",
        "pwd_change_date":"None",
        "PAN":"XXXXXXXXXX",
        "mobile_number":"9XXXXX678",
        "totp":True,
        "pwd_to_expire":90
        "ddpi_enabled": False,
        "mtf_enabled": False
    }
  }
```


Funds

Shows the balance available for the user for capital as well as the commodity market.

Response Attributes

Attribute	Data Type	Description
`id`	int	Unique identity for particular fund
`title`	string	Each title represents a heading of the ledger
`equityAmount`	float	The amount in the capital ledger for the above-mentioned title
`commodityAmount`	float	The amount in the commodity ledger for the above-mentioned title

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

# Make a request to get the funds information
response = fyers.funds()
print(response)

--------------------------------------------------------------------------------
Sample Response
--------------------------------------------------------------------------------

  {
    "code":200,
    "message":"",
    "s":"ok",
    "fund_limit":[
        {
          "id":1,
          "title":"Total Balance",
          "equityAmount":58.150000000000006,
```

```
    "commodityAmount":0
  },
  {
    "id":2,
    "title":"Utilized Amount",
    "equityAmount":0.3,
    "commodityAmount":0
  },
  {
    "id":3,
    "title":"Clear Balance",
    "equityAmount":58.150000000000006,
    "commodityAmount":0
  },
  {
    "id":4,
    "title":"Realized Profit and Loss",
    "equityAmount":-0.3,
    "commodityAmount":0
  },
  {
    "id":5,
    "title":"Collaterals",
    "equityAmount":0,
    "commodityAmount":0
  },
  {
    "id":6,
    "title":"Fund Transfer",
    "equityAmount":0,
    "commodityAmount":0
  },
  {
    "id":7,
    "title":"Receivables",
    "equityAmount":0,
    "commodityAmount":0
  },
  {
    "id":8,
    "title":"Adhoc Limit",
    "equityAmount":0,
    "commodityAmount":0
  },
  {
    "id":9,
    "title":"Limit at start of the day",
    "equityAmount":58.45,
```

```
      "commodityAmount":0
    },
    {
      "id":10,
      "title":"Available Balance",
      "equityAmount":58.150000000000006,
      "commodityAmount":0
    }
  ]
}
```


Holdings

Fetches the equity and mutual fund holdings which the user has in this demat account. This will include T1 and demat holdings.

Request Attributes - For each holding

Attribute	Data Type	Description
`symbol`	string	Eg: NSE:RCOM-EQ
`holdingType`	string	Identify the type of holding View Details
`quantity`	int	The quantity of the symbol which the user has at the beginning of the day
`remainingQuantity`	int	This reflects the quantity - the quantity sold during the day
`pl`	float	Profit and loss made
`costPrice`	float	The original buy price of the holding
`marketVal`	float	The Market value of the current holding
`ltp`	float	LTP is the price from which the next sale of the stocks happens
`id`	int	The unique value for each holding
`fytoken`	string	Fytoken is a unique identifier for every symbol. View Details

Attribute	Data Type	Description
`exchange`	int	The exchange in which order is placed. View Details
`segment`	int	The segment in which the holding is taken. View Details
`isin`	string	Unique ISIN provided by exchange for each symbol
`qty_t1`	int	Quantity t+1 day
`remainingPledgeQuantity`	int	Remaining Pledged quantity
`collateralQuantity`	int	Pledged quantity

Response Attributes - Overall holdings

Attribute	Data Type	Description
`count_total`	int	Total number of holdings present
`total_investment`	float	Invested amount for the current holdings
`total_current_value`	float	The present value of the holdings
`total_pl`	float	Total profit and loss made
`pnl_perc`	float	Percentage value of the total pnl

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

response = fyers.holdings()
```

```
print(response)

--------------------------------------------------------------------------------
  Sample Success Response
--------------------------------------------------------------------------------

  {
    "code":200,
    "message":"",
    "s":"ok",
    "overall":{
        "count_total":1,
        "pnl_perc":21.9136,
        "total_current_value":23.700000000000003,
        "total_investment":19.44,
        "total_pl":4.26
    },
    "holdings":[
        {
          "costPrice":6.48,
          "id":0,
          "fyToken":"101000000014366",
          "symbol":"NSE:IDEA-EQ",
          "isin":"INE669E01016",
          "quantity":3,
          "exchange":10,
          "segment":10,
          "qty_t1":0,
          "remainingQuantity":3,
          "collateralQuantity":0,
          "remainingPledgeQuantity":3,
          "pl":4.26,
          "ltp":7.9,
          "marketVal":23.700000000000003,
          "holdingType":"HLD"
        }
    ]
  }
```

Logout

This invalidates the access token, revoking it only for the specific app without affecting other active apps or web and mobile sessions.

Request samples

cURL

Copy

```
Curl Request Method
curl -H "Authorization: app_id:access_token" POST 'https://api-t1.fyers.in/ap

-------------------------------------------------------------------------------
Sample Response
-------------------------------------------------------------------------------
  {
    "s": "ok",
    "code": 200,
    "message": "you are successfully logged out",
  }
```

Transaction Info

Orders

Fetches all the orders placed by the user across all platforms and exchanges in the current trading day.

Response attributes - For each order

Attribute	Data Type	Description
`id`	string	The unique order id assigned for each order
`exchOrdId`	string	The order id provided by the exchange
`symbol`	string	The symbol for which order is placed

Attribute	Data Type	Description
`qty`	int	The original order qty
`remainingQuantity`	int	The remaining qty
`filledQty`	int	The filled qty after partial trades
`status`	int	1 => Canceled 2 => Traded / Filled 3 => (Not used currently) 4 => Transit 5 => Rejected 6 => Pending 7 => Expired View Details
`slNo`	int	This is used to sort the orders based on the time
`message`	string	The error messages are shown here
`segment`	int	10 => E (Equity) 11 => D (F&O) 12 => C (Currency) 20 => M (Commodity) View Details
`limitPrice`	float	The limit price for the order
`stopPrice`	float	The limit price for the order
`productType`	string	The product type
`type`	int	1 => Limit Order 2 => Market Order 3 => Stop Order (SL-M) 4 => Stoplimit Order (SL-L)
`side`	int	1 => Buy -1 => Sell View Details
`disclosedQty`	int	Disclosed quantity
`orderValidity`	string	DAY IOC
`orderDateTime`	string	The order time as per DD-MMM-YYYY hh:mm:ss in IST
		The parent order id will be provided only for applicable

Attribute	Data Type	Description
`parentId`	string	orders.. Eg: BO Leg 2 & 3 and CO Leg 2
`tradedPrice`	float	The average traded price for the order
`source`	string	Source from where the order was placed. View Details
`fytoken`	string	Fytoken is a unique identifier for every symbol. View Details
`offlineOrder`	boolean	False => When market is open True => When placing AMO order
`pan`	string	PAN of the client
`clientId`	string	The client id of the fyers user
`exchange`	int	The exchange in which order is placed View Details
`instrument`	int	Exchange instrument type View Details
`discloseQty`	int	Disclosed quantity
`orderTag`	string	ordertag provided when placing the order **Note**: `1:` will be concatenated at the start of tag provided by user. `2:` will be concatenated at the start of tag generated internally by Fyers. Default value if tag not provided when order is placed is `2:Untagged`

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"
```

```python
# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

response = fyers.orderbook()
print(response)
```

Sample Success Response

```
  {
    "code":200,
    "message":"",
    "s":"ok",
    "orderBook":[
        {
          "clientId":"XXXXX86",
          "exchange":10,
          "fyToken":"101000000014366",
          "id":"23080444447604",
          "offlineOrder":false,
          "source":"W",
          "status":2,
          "type":2,
          "pan":"",
          "limitPrice":8.1,
          "productType":"INTRADAY",
          "qty":1,
          "disclosedQty":0,
          "remainingQuantity":0,
          "segment":10,
          "symbol":"NSE:IDEA-EQ",
          "description":"VODAFONE IDEA LIMITED",
          "ex_sym":"IDEA",
          "orderDateTime":"02-Aug-2023 13:01:42",
          "side":1,
          "orderValidity":"DAY",
          "stopPrice":0,
          "tradedPrice":8.1,
          "filledQty":1,
          "exchOrdId":"1100000024706527",
          "message":"",
          "ch":-0.35,
          "chp":-4.24,
          "lp":7.9,
          "orderNumStatus":"23080444447604:2",
          "slNo":1,
```

```
            "orderTag": "1:Ordertag"
        }
    ]
}
```


Order Filter By Order Id

You can query for a particular order id by passing the id in the get parameters

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

orderId = "23080444447604"
data = {"id":orderId}

response = fyers.orderbook(data=data)
print(response)

-----------------------------------------------------------------
Sample Success Response
-----------------------------------------------------------------

  {
    "code":200,
    "message":"",
    "s":"ok",
    "orderBook": [{
              "clientId":"XXXXX86",
```

```
            "exchange":10,
            "fyToken":"101000000014366",
            "id":"23080444447604",
            "offlineOrder":false,
            "source":"W",
            "status":2,
            "type":2,
            "pan":"",
            "limitPrice":8.1,
            "productType":"INTRADAY",
            "qty":1,
            "disclosedQty":0,
            "remainingQuantity":0,
            "segment":10,
            "symbol":"NSE:IDEA-EQ",
            "description":"VODAFONE IDEA LIMITED",
            "ex_sym":"IDEA",
            "orderDateTime":"02-Aug-2023 13:01:42",
            "side":1,
            "orderValidity":"DAY",
            "stopPrice":0,
            "tradedPrice":8.1,
            "filledQty":1,
            "exchOrdId":"1100000024706527",
            "message":"",
            "ch":-0.35,
            "chp":-4.24,
            "lp":7.9,
            "orderNumStatus":"23080444447604:2",
            "slNo":1,
            "orderTag": "1:Ordertag"
        }]
}
```


Order Filter By Order tag

You can query for a particular orderTag by passing the orderTag in the get parameters

Request samples

cURL

Copy

```
Curl Request Method
curl -H "Authorization:app_id:access_token"  https://api-t1.fyers.in/api/v3/o

------------------------------------------------------------------
Sample Success Response
------------------------------------------------------------------
{
  "s": "ok",
  "code": 200,
  "message": "",
  "orderBook": [{
      "clientId": "X******",
      "id": "23030900015105",
      "exchOrdId": "1100000001089341",
      "qty": 1,
      "remainingQuantity": 0,
      "filledQty": 1,
      "discloseQty": 0,
      "limitPrice": 6.95,
      "stopPrice": 0,
      "tradedPrice": 6.95,
      "type": 1,
      "fyToken": "101000000014366",
      "exchange": 10,
      "segment": 10,
      "symbol": "NSE:IDEA-EQ",
      "instrument": 0,
      "message": "",
      "offlineOrder": False,
      "orderDateTime": "09-Mar-2023 09:34:38",
      "orderValidity": "DAY",
      "pan": "",
      "productType": "CNC",
      "side": -1,
      "status": 2,
      "source": "W",
      "ex_sym": "IDEA",
      "description": "VODAFONE IDEA LIMITED",
      "ch": -0.1,
      "chp": -1.44,
      "lp": 6.85,
      "slNo": 1,
      "dqQtyRem": 0,
      "orderNumStatus": "23030900015105:2",
```

```
        "disclosedQty": 0,
        "orderTag": "1:Ordertag"
    }]
}
```


Positions

Fetches the current open and closed positions for the current trading day. Note that previous trading day's closed positions will not be shown here.

Response attributes - For each position

Attribute	Data Type	Description
`symbol`	string	Eg: NSE:SBIN-EQ
`id`	string	The unique value for each position
`buyAvg`	float	Average buy price
`buyQty`	int	Total buy qty
`sellAvg`	float	Average sell price
`sellQty`	int	Total sell qty
`netAvg`	float	netAvg
`netQty`	int	Net qty
`side`	int	The side shows whether the position is long / short View Details
`qty`	int	Absolute value of net qty
`productType`	string	The product type of the position View Details
`realized_profit`	float	The realized p&l of the position
`pl`	float	The total p&l of the position
`crossCurrency`	string	Y => It is cross currency position N => It is not a cross currency position
`rbiRefRate`	float	Incase of cross currency position, the rbi reference rate will be required to calculate the p&l

Attribute	Data Type	Description
`qtyMulti_com`	float	Incase of commodity positions, this multiplier is required for p&l calculation
`segment`	int	The segment in which the position is taken. View Details
`exchange`	int	The exchange in which the position is taken. View Details
`slNo`	int	This is used for sorting of positions
`ltp`	float	LTP is the price from which the next sale of the stocks happens
`fytoken`	string	Fytoken is a unique identifier for every symbol. View Details
`cfBuyQty`	int	Carry forward buy quantity
`cfSellQty`	int	Carry forward sell quantity
`dayBuyQty`	int	Day forward buy quantity
`daySellQty`	int	Day forward sell quantity
`exchange`	int	The exchange in which order is placed View Details

Response attributes - Overall Positions

Attribute	Data Type	Description
`count_total`	int	Total number of positions present
`count_open`	int	Total number of positions opened
`pl_total`	float	Total profit and losses
`pl_realized`	float	Profit and losses when the owned product is sold
`pl_unrealized`	float	Profit and loses when the product is owned , but is not sold

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

response = fyers.positions()
print(response)
```

Sample Success Response

```
{
    "s": "ok",
    "code": 200,
    "message": "",
    "netPositions":
                [{
                  "netQty":1,
                  "qty":1,
                  "avgPrice":72256.0,
                  "netAvg": 71856.0,
                  "side":1,
                  "productType":"MARGIN",
                  "realized_profit":400.0,
                  "unrealized_profit":461.0,
                  "pl":861.0,
                  "ltp":72717.0,
                  "buyQty":2,
                  "buyAvg":72256.0,
                  "buyVal":144512.0,
                  "sellQty":1,
                  "sellAvg":72656.0,
                  "sellVal":72656.0,
                  "slNo":0,
                  "fyToken":"1120200831217406",
                  "crossCurrency":"N",
                  "rbiRefRate":1.0,
                  "qtyMulti_com":1.0,
                  "segment":20,
                  "symbol":"MCX:SILVERMIC20AUGFUT",
                  "id":"MCX:SILVERMIC20AUGFUT-MARGIN",
                  "cfBuyQty": 0,
```

```
                "cfSellQty": 0,
                "dayBuyQty": 0,
                "daySellQty": 1,
                "exchange": 10,
            }],
    "overall":
            {
                "count_total":1,
                "count_open":1,
                "pl_total":861.0,
                "pl_realized":400.0,
                "pl_unrealized":461.0
            }
}
```


Trades

Fetches all the trades for the current day across all platforms and exchanges in the current trading day.

Response attributes - For each trade

Attribute	Data Type	Description
`symbol`	string	Eg: NSE:SBIN-EQ
`row`	int	The unique value to sort the trades
`orderDateTime`	string	The time when the trade occured in “DD-MM-YYYY hh:mm:ss” format in IST
`orderNumber`	string	The order id for which the trade occurred
`tradeNumber`	string	The trade number generated by the exchange
`tradePrice`	float	The traded price
`tradeValue`	float	The total traded value
`tradedQty`	int	The total traded qty
`side`	int	1 => Buy -1 => Sell View Details

Attribute	Data Type	Description
`productType`	string	The product in which the order was placed View Details
`exchangeOrderNo`	string	The order number provided by the exchange
`segment`	int	The segment in which order is placed View Details
`exchange`	int	The exchange in which order is placed View Details
`fyToken`	string	Fytoken is a unique identifier for every symbol.
`orderTag`	string	ordertag provided when placing the order **Note**: `1:` will be concatenated at the start of tag provided by user. `2:` will be concatenated at the start of tag generated internally by Fyers. Default value if tag not provided when order is placed is `1:Untagged`

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

response = fyers.tradebook()
print(response)

--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------
{
  "s": "ok",
```

```
"code": 200,
"message": "",
"tradeBook":
            [{
              "clientId":"FXXXXX",
              "orderDateTime":"07-Aug-2020 13:51:12",
              "orderNumber":"120080789075",
              "exchangeOrderNo": "1200000009204725",
              "exchange":10,
              "side":1,
              "segment":10,
              "orderType":2,
              "fyToken":"101000000010666",
              "productType":"CNC",
              "tradedQty":10,
              "tradePrice":32.7,
              "tradeValue":327.0,
              "tradeNumber":"52605023",
              "row":52605023,
              "symbol":"NSE:PNB-EQ"
              "orderTag": "1:Ordertag"
            },
            {
              "clientId":"FXXXXX",
              "orderDateTime":"07-Aug-2020 13:48:12",
              "orderNumber":"120080789139",
              "exchangeOrderNo": "1000000012031528",
              "exchange":10,
              "side":1,
              "segment":10,
              "orderType":2,
              "fyToken":"101000000010454",
              "productType":"CNC",
              "tradedQty":19,
              "tradePrice":14.1,
              "tradeValue":267.9,
              "tradeNumber":"3281523",
              "row":3281523,
              "symbol":"NSE:CENTRUM-EQ"
              "orderTag": "1:Ordertag"
            },
            {
              "clientId":"FXXXXX1",
              "orderDateTime":"07-Aug-2020 13:47:22",
              "orderNumber":"120080797993",
              "exchangeOrderNo": "1100000008047027",
              "exchange":10,
              "side":1,
```

```
                "segment":10,
                "orderType":2,
                "fyToken":"101000000018783",
                "productType":"CNC",
                "tradedQty":4,
                "tradePrice":115.5,
                "tradeValue":462.0,
                "tradeNumber":"27945307",
                "row":27945307,
                "symbol":"NSE:IDFNIFTYET-EQ"
                "orderTag": "1:Ordertag"
              }]
}
```


Trades Filter By Order tag

You can query for a particular orderTag by passing the orderTag in the get parameters.

Request samples

cURL | C# | Java

Copy

```
Curl Request Method
curl -H "Authorization: app_id:access_token" https://api-t1.fyers.in/api/v3/t

------------------------------------------------------------------------
Sample Success Response
------------------------------------------------------------------------
{
  "s": "ok",
  "code": 200,
  "message": "",
  "tradeBook":
              [{
                "clientId":"FXXXXX",
                "orderDateTime":"07-Aug-2020 13:51:12",
                "orderNumber":"120080789075",
```

```
    "exchangeOrderNo": "1200000009204725",
    "exchange":10,
    "side":1,
    "segment":10,
    "orderType":2,
    "fyToken":"101000000010666",
    "productType":"CNC",
    "tradedQty":10,
    "tradePrice":32.7,
    "tradeValue":327.0,
    "tradeNumber":"52605023",
    "row":52605023,
    "symbol":"NSE:PNB-EQ",
    "orderTag": "1:Ordertag"
  },
  {
    "clientId":"FXXXXX",
    "orderDateTime":"07-Aug-2020 13:48:12",
    "orderNumber":"120080789139",
    "exchangeOrderNo": "1000000012031528",
    "exchange":10,
    "side":1,
    "segment":10,
    "orderType":2,
    "fyToken":"101000000010454",
    "productType":"CNC",
    "tradedQty":19,
    "tradePrice":14.1,
    "tradeValue":267.9,
    "tradeNumber":"3281523",
    "row":3281523,
    "symbol":"NSE:CENTRUM-EQ",
    "orderTag": "1:Ordertag"
  },
  {
    "clientId":"FXXXXX1",
    "orderDateTime":"07-Aug-2020 13:47:22",
    "orderNumber":"120080797993",
    "exchangeOrderNo": "1100000008047027",
    "exchange":10,
    "side":1,
    "segment":10,
    "orderType":2,
    "fyToken":"101000000018783",
    "productType":"CNC",
    "tradedQty":4,
    "tradePrice":115.5,
    "tradeValue":462.0,
```

```
                "tradeNumber":"27945307",
                "row":27945307,
                "symbol":"NSE:IDFNIFTYET-EQ",
                "orderTag": "1:Ordertag"
            }]
}
```


Order Placement Guide

Order Placement Guide

The order placement process requires you to carefully input several parameters at the time of making the request. There are several validations which are required to be done before sending the request.

Order Types

- **Limit Order**
  - type: 1
  - A limit order allows you to buy or sell an asset at a specific price (limitPrice) or better. It will only be executed at the specified price or lower for a buy order, and at the specified price or higher for a sell order.

  Sample input:


```
{ "symbol": "NSE:SBIN-EQ", "qty": 100, "type": 1, "side": 1, "productType":
```


- **Market Order**
  - type: 2
  - A market order allows you to buy or sell an asset at the current market price. The limitPrice and stopPrice should be set to 0, as it is not used in this order type.

Sample input:

```
{ "symbol": "NSE:SBIN-EQ", "qty": 100, "type": 2, "side": 1, "productType":
```

- **Stop Order / SL-M**
  - type: 3
  - A stop order is designed to limit losses on a position. It becomes a market order when the stopPrice is reached. The stopPrice is the trigger price at which the market order will be placed.
  - The stopPrice must not be higher than the Last Traded Price (ltp) for a sell order and not lower than the ltp for a buy order.

Sample input:

```
{ "symbol": "NSE:SBIN-EQ", "qty": 100, "type": 3, "side": 1, "productType":
```

- **Stop Limit Order / SL-L**
  - type: 4
  - A stop-limit order combines features of a stop order and a limit order. It triggers a limit order once the stopPrice is reached.
  - The stopPrice must not be higher than the Last Traded Price (ltp) for a sell order and not lower than the ltp for a buy order.
  - The stopPrice must be lower than the limitPrice for a buy order and higher than the limitPrice for a sell order.

Sample input:

```
{ "symbol": "NSE:SBIN-EQ", "qty": 100, "type": 4, "side": 1, "productType":
```

Product Types

- **Intraday**
  - Used to place orders which will be bought and sold the same day
  - Order type can be anything (Market, Limit, Stop, and Stop Limit)

- **CNC**
    - Used to place orders only in stocks which will be carried forward
    - Order type can be anything (Market, Limit, Stop, and Stop Limit)
- **Margin**
    - Used to place orders in derivative contracts which will be carried forward
    - Order type can be anything (Market, Limit, Stop, and Stop Limit)
- **Cover Order (CO)**
    - stopLoss is a mandatory input
    - stopLoss price is given in points denominated. (Eg: SBIN LTP = 300. For the above example, for buy CO, if you want to put stop loss as 298 then you have to pass "stopLoss" value as 2 (difference between ltp and your desired stop loss))
      Note:
      1. "stopLoss" value can be float but should be multiple of 0.0500.
      2. Now let's say if you don't provide stopLoss value as multiple of 0.0500 then you can find this error message {'code': -50, 'message': 'StopLoss not a multiple of tick size 0.0500', 's': 'error'}
    - The order type can be either market or limit (If the market order, then the stop loss price should not be higher than the ltp in buy and lower than the ltp for sell. If the limit order, then the stop loss price should not be higher than the limit price in buy and lower than the limit for sell)
    - Validity should be “DAY.”
    - Disclosed quantity should be 0.
- **Bracket Order (BO)**
    - stopLoss is a mandatory input
    - takeProfit is a mandatory input
    - The stopLoss and takeProfit are denominated in rupees above and below the trade price. (Eg: SBIN LTP = 300. So the user can give a target of Rs. 10 and stop loss or Rs. 5.)
    - Order type can be either market, limit, stop, or stop limit
    - Validity should be “DAY.”
    - Disclosed quantity should be 0
- **Margin Trading Facility (MTF)**
    - This order type is for placing trades with additional margin.
    - MTF orders require activation.
    - Only stocks from the approved MTF list are allowed.
    - Supported order types: Market, Limit, Stop, and Stop Limit.

Price Validations

For all transaction requests (Order placement / modification), the price input should be in accordance with the exchange provided tick size for the particular symbol.

- Tick size is applicable for all price inputs (Limit / stop / stopLoss / target)
- Each symbol will have its own tick size
- All price inputs have to be in multiples of the tick size
- The tick size is available in the symbol master

Quantity Validations

For all transaction requests (Order placement / modification), the quantity input should be in accordance with the exchange provided minimum lot size.

Segment	Minimum Quantity Details
Equities	The quantity will be in multiples of 1
Equity Derivatives	The quantity will be in multiples of the lot size specified by the exchange
Currency Derivatives	The quantity will be in multiples of 1
Commodity Derivatives	The quantity will be in multiples of 1

Order Tag

For all Order placement requests where tags are passed they should meet following requirements.

- Ordertag string should be Alphanumeric i.e. no space or special characters will be allowed.
- Ordertag string should not exceed length of 30 characters and should have minimum length of 1 character.
- Ordertag string cannot be your **clientID** or string **Untagged**.
- Ordertagging is currently not supported for ProductType **BO** & **CO**. Orders with ordertag for CO/BO product Type will be rejeceted.

Order Placement

Single Order

This allows the user to place an order to any exchange via Fyers

Request attributes

Attribute	Data Type	Description
`symbol*`	string	Eg: NSE:SBIN-EQ
`qty*`	int	The quantity should be in multiples of lot size for derivatives.
`type*`	int	1 => Limit Order 2 => Market Order 3 => Stop Order (SL-M) 4 => Stoplimit Order (SL-L) View Details
`side*`	int	1 => Buy -1 => Sell View Details
`productType*`	string	CNC => For equity only INTRADAY => Applicable for all segments. MARGIN => Applicable only for derivatives CO => Cover Order BO => Bracket Order MTF => Approved Symbols Only View Details
`limitPrice*`	float	Default => 0 Provide valid price for Limit and Stoplimit orders
`stopPrice*`	float	Default => 0 Provide valid price for Stop and Stoplimit orders
`disclosedQty*`	int	Default => 0 Allowed only for Equity
`validity*`	string	IOC => Immediate or Cancel DAY => Valid till the end of the day
`offlineOrder*`	boolean	False => When market is open True => When placing AMO order
`stopLoss*`	float	Default => 0 Provide valid price for CO and BO orders

Attribute	Data Type	Description
`takeProfit*`	float	Default => 0 Provide valid price for BO orders
`orderTag`	string	(Optional) Tag you want to assign to the specific order

Response attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	This is the code to identify specific responses If 201 comes in code it implies that order request has been made but no acknowledgement has been received in this case check orderbook before placing order
`message`	string	Message to clarify the request status. Eg: Order submitted successfully. Your Order Ref. No.119062829763
`id`	string	The order number of the placed order. Eg: 119031547242

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"
# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

data = {
    "symbol":"NSE:IDEA-EQ",
    "qty":1,
    "type":2,
    "side":1,
    "productType":"INTRADAY",
```

```
    "limitPrice":0,
    "stopPrice":0,
    "validity":"DAY",
    "disclosedQty":0,
    "offlineOrder":False,
    "orderTag":"tag1"
}
response = fyers.place_order(data=data)
print(response)
-----------------------------------------------------------------------------
Sample Success Response
-----------------------------------------------------------------------------
  {
    "s": "ok",
    "code": 1101,
    "message": "Order submitted successfully. Your Order Ref. No.808058117761
    "id":"808058117761"
  }
```

Multi Order

You can place upto 10 orders simultaneously via the API.
While Placing Multi orders you need to pass an ARRAY containing the orders request attributes

Sample Request :

```
[{ "symbol":"MCX:SILVERM20NOVFUT", "qty":1, "type":1, "side":1, "productType":"
```

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
```

```
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"
# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

data = [{
    "symbol":"NSE:SBIN-EQ",
    "qty":1,
    "type":2,
    "side":1,
    "productType":"INTRADAY",
    "limitPrice":0,
    "stopPrice":0,
    "validity":"DAY",
    "disclosedQty":0,
    "offlineOrder":False,
},
{
    "symbol":"NSE:IDEA-EQ",
    "qty":1,
    "type":2,
    "side":1,
    "productType":"INTRADAY",
    "limitPrice":0,
    "stopPrice":0,
    "validity":"DAY",
    "disclosedQty":0,
    "offlineOrder":False,
},{
    "symbol":"NSE:SBIN-EQ",
    "qty":1,
    "type":2,
    "side":1,
    "productType":"INTRADAY",
    "limitPrice":0,
    "stopPrice":0,
    "validity":"DAY",
    "disclosedQty":0,
    "offlineOrder":False,
},
{
    "symbol":"NSE:IDEA-EQ",
    "qty":1,
    "type":2,
    "side":1,
    "productType":"INTRADAY",
    "limitPrice":0,
    "stopPrice":0,
    "validity":"DAY",
```

```
    "disclosedQty":0,
    "offlineOrder":False,
}]

response = fyers.place_basket_orders(data=data)
print(response)

--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------
   {
        "s":"ok",
        "code":200,
        "message":"",
        "data":[{"statusCode":200,
              "body":{
                        "s":"ok",
                        "code":1101,
                        "message":"Order submitted successfully. Your Order R
                        "id":"52104097619"
                    },
        "statusDescription":"HTTP OK"
        },
        {
        "statusCode":200,
        "body":{
        "s":"ok",
        "code":1101,
        "message":"Order submitted successfully. Your Order Ref. No.521040976
        "id":"52104097620"
        },
          "statusDescription":"HTTP OK"
        },
        {
        "statusCode":200,
        "body":{
        "s":"ok",
        "code":1101,
        "message":"Order submitted successfully. Your Order Ref. No.521040976
        "id":"52104097621"
        },
        "statusDescription":"HTTP OK"
        },
        {
        "statusCode":400,
        "body":{
          "s":"error",
          "code":-392,
```

```
            "message":"Price should be in multiples of tick size.",
            "id":""
            },
        "statusDescription":"400 Bad Request"
    }]
}
```


MultiLeg Order

This allows the user to place an MultiLeg order to NSE via Fyers.
Note: As per exchange mandate, while selecting symbols please check the "stream" key in the symbol master JSON file to ensure both legs belong to the same stream.

Request attributes

Attribute	Data Type	Description
`orderTag`	string	(Optional) Tag you want to assign to the specific order
`productType*`	string	INTRADAY => Applicable for NFO segments. MARGIN => Applicable for NFO segments.
`offlineOrder*`	boolean	False => When market is open True => When placing AMO order.
`orderType*`	string	3L => Applicable for 3 leg order. 2L => Applicable for 2 leg order.
`validity*`	string	IOC => Immediate or Cancel
`legs*`	json	It represents multiple trading legs in a multi-leg order. Each leg within the legs object corresponds to a specific trading instrument and order details. Each leg is identified by a unique key (leg1, leg2, leg3)

Leg attributes

Attribute	Data Type	Description
`symbol*`	string	Eg: NSE:SBIN24JUNFUT
`qty*`	int	The quantity should be in multiples of lot size for derivatives.

Attribute	Data Type	Description
`side*`	int	1 => Buy -1 => Sell
`type*`	int	1 => Limit Order
`limitPrice*`	float	Provide valid price for Limit orders

Response attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	This is the code to identify specific responses If 201 comes in code it implies that order request has been made but no acknowledgement has been received in this case check orderbook before placing order
`message`	string	Message to clarify the request status. Eg: Order submitted successfully. Your Order Ref. No.119062829763
`id`	string	The order number of the placed order. Eg: 119031547242

Input Validations

Error Message	Description
`The input symbol is invalid.`	Input symbol ticker is not found in the exchange data
`Only NFO symbols are allowed`	Options and Future symbols listed in NSE are allowed.
`The input symbol is banned for FNO trading.`	The input symbol is banned for trading by the exchange.
`All symbols must be related to the same underlying symbol.`	All symbols must be related to same

Error Message	Description
	symbol
`Quantity should be multiples of the lot size.`	Input qty(Quantity) should be multiples of minimum lot size of that symbol
`You cannot add same symbol for two legs.`	Input data should not have same symbol in more than one leg.
`Group combination not found`	Input Symbols do not belong to same stream group.

Request samples

cURL | Python | Node | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"
# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

data = {
    "orderTag": "tag1",
    "productType": "MARGIN",
    "offlineOrder": False,
    "orderType": "3L",
    "validity": "IOC",
    "legs": {
        "leg1": {
          "symbol": "NSE:SBIN24JUNFUT",
          "qty": 750,
          "side": 1,
          "type": 1,
          "limitPrice": 800
        },
        "leg2": {
```

```
            "symbol": "NSE:SBIN24JULFUT",
            "qty": 750,
            "side": 1,
            "type": 1,
            "limitPrice": 800
        },
        "leg3": {
            "symbol": "NSE:SBIN24JUN900CE",
            "qty": 750,
            "side": 1,
            "type": 1,
            "limitPrice": 3
        }
    }
}
response = fyers.place_multileg_order(data=data)
print(response)
--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------
  {
    "s": "ok",
    "code": 1101,
    "message": "Successfully placed order",
    "id":"808058117761"
  }
```

GTT Orders

GTT (Good Till Trigger) is used to create orders with longer validity. It helps you set a Target or Stop-Loss for your open positions or holdings, and you can also use it to take new positions. The order remains active for up to one year.

GTT Single

This type is used to create a single GTT order.

- It supports placing orders for CNC, Margin, and MTF.
- It can be applied either as a Target or Stop Loss for your existing holdings/positions.
- Alternatively, it can be used to create a GTT order for a new position/holding.

Request attributes

Attribute	Data Type	Description
`id*`	string	Unique identifier for the order to be modified, e.g., "25010700000001".
`Side`	integer	Indicates the side of the order: `1` for buy, `-1` for sell.
`symbol*`	string	The instrument's unique identifier, e.g., "NSE:CHOLAFIN-EQ"
`productType*`	string	The product type for the order. Valid values: "CNC", "MARGIN", "MTF".
`orderInfo*`	object	Contains information about the GTT/OCO order legs.
`orderInfo.leg1*`	object	Details for GTT order leg. Mandatory for all orders.
`orderInfo.leg1.price*`	number	Price at which the order
`orderInfo.leg1.triggerPrice`	number	Trigger price for the GTT order. NOTE: for OCO order this leg trigger price should be always above LTP
`orderInfo.leg1.qty*`	integer	Quantity for the GTT order leg.
`orderInfo.leg2*`	object	Details for OCO order leg. Optional and included only for OCO orders.
`orderInfo.leg2.price*`	number	Price at which the second leg of the OCO order should be placed.
`orderInfo.leg2.triggerPrice*`	number	Trigger price for the second leg of the OCO order. NOTE: for OCO order this leg trigger price should be always below LTP
`orderInfo.leg2.qty*`	integer	Quantity for the second leg of the OCO order..

Response attributes

Attribute	Data Type	Description
`code`	integer	Status code indicating the outcome of the request: - `1101`: Order placed successfully. - `201`: Order is in transit. Confirm details using the Orderbook API. - Other codes represent specific error conditions.
`message*`	string	Descriptive message about the request status, e.g., "Successfully placed order".
`s*`	string	Request status indicator: "ok" for success, "error" for failure.
`id*`	string	Unique identifier for the placed order, returned when the order is successfully processed.

Request samples

cURL | Python | Node | Web Modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"
# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

data = {
      "side": 1,
      "symbol": "NSE:SBIN-EQ",
      "productType":"CNC",
      "orderInfo": {
          "leg1": {
              "price": 1000,
              "triggerPrice": 1000,
              "qty": 3
          }
      }
  }
response = fyers.place_gtt_order(data=data)
print(response)
--------------------------------------------------------------------------------
Sample Success Response
```

```
--------------------------------------------------------------------------------
  {
    "code": 1101,
    "message": "Successfully placed order",
    "s": "ok",
    "id":"25012400002099"
  }
```

GTT OCO

This type allows you to place both a Stop Loss and a Target order simultaneously. If one order is triggered, the other gets automatically canceled.

To avoid rejections:

- For Leg1: The trigger price must always be greater than the LTP
- For Leg 2: The trigger price must always be less than the LTP

Request attributes

Attribute	Data Type	Description
`id*`	string	Unique identifier for the order to be modified, e.g., "25010700000001".
`Side`	integer	Indicates the side of the order: `1` for buy, `-1` for sell.
`symbol*`	string	The instrument's unique identifier, e.g., "NSE:CHOLAFIN-EQ"
`productType*`	string	The product type for the order. Valid values: "CNC", "MARGIN", "MTF".
`orderInfo*`	object	Contains information about the GTT/OCO order legs.
`orderInfo.leg1*`	object	Details for GTT order leg. Mandatory for all orders.
`orderInfo.leg1.price*`	number	Price at which the order
`orderInfo.leg1.triggerPrice`	number	Trigger price for the GTT order. NOTE: for OCO order this leg trigger price should be always above LTP

Attribute	Data Type	Description
`orderInfo.leg1.qty*`	integer	Quantity for the GTT order leg.
`orderInfo.leg2*`	object	Details for OCO order leg. Optional and included only for OCO orders.
`orderInfo.leg2.price*`	number	Price at which the second leg of the OCO order should be placed.
`orderInfo.leg2.triggerPrice*`	number	Trigger price for the second leg of the OCO order. NOTE: for OCO order this leg trigger price should be always below LTP
`orderInfo.leg2.qty*`	integer	Quantity for the second leg of the OCO order..

Response attributes

Attribute	Data Type	Description
`code`	integer	Status code indicating the outcome of the request: - `1101`: Order placed successfully. - `201`: Order is in transit. Confirm details using the Orderbook API. - Other codes represent specific error conditions.
`message*`	string	Descriptive message about the request status, e.g., "Successfully placed order".
`s*`	string	Request status indicator: "ok" for success, "error" for failure.
`id*`	string	Unique identifier for the placed order, returned when the order is successfully processed.

Request samples

cURL | Python | Node | Web Modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel
```

```
client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"
# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

data = {
      "side": 1,
      "symbol": "NSE:SBIN-EQ",
      "productType":"CNC",
      "orderInfo": {
          "leg1": {
              "price": 1000,
              "triggerPrice": 1000,
              "qty": 3
          },
          "leg2": {
              "price": 600,
              "triggerPrice": 600,
              "qty": 3
          }
      }
  }
response = fyers.place_gtt_order(data=data)
print(response)
--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------
  {
    "code": 1101,
    "message": "Successfully placed order",
    "s": "ok",
    "id":"25012400002100"
  }
```

GTT Modify Order

This allows the user to modify pending GTT orders. Users can provide parameters which need to be modified. In case a particular parameter has not been provided, the original value will be considered.

Request attributes

Attribute	Data Type	Description
`id*`	string	Unique identifier for the order to be modified, e.g., "25010700000001".
`orderInfo*`	object	Contains updated information about the GTT/OCO order legs.
`orderInfo.leg1*`	object	Details for GTT order leg. Mandatory for all modifications.
`orderInfo.leg1.price*`	number	Updated price at which the order should be placed.
`orderInfo.leg1.triggerPrice*`	number	Updated trigger price for the GTT order. NOTE: for OCO order this leg trigger price should be always above LTP.
`orderInfo.leg1.qty*`	integer	Updated quantity for the GTT order leg.
`orderInfo.leg2*`	object	Details for OCO order leg. Required if the order is an OCO type.
`orderInfo.leg2.price*`	number	Updated price for the second leg of the OCO order.
`orderInfo.leg2.triggerPrice*`	number	Updated trigger price for the second leg of the OCO order. NOTE: for OCO order this leg trigger price should be always below LTP.
`orderInfo.leg2.qty*`	integer	Updated quantity for the second leg of the OCO order.

Response attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	Status code indicating the outcome of the modification request: - 1102: Order modification successful. - 201: Order modification is in transit. Confirm details using the Orderbook API. - Other codes represent specific error conditions.
`message`	string	Descriptive message about the modification request. e.g., "Successfully modified order".

Attribute	Data Type	Description
`id`	string	Unique identifier for the modified order, returned if the modification is successful.

Request samples

cURL | Python | Node | Web Modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

data = {
    "id": "25012400002074",
    "orderInfo": {
        "leg1": {
            "price": 1020,
            "triggerPrice": 1020,
            "qty": 5
        },
        "leg2": {
            "price": 620,
            "triggerPrice": 620,
            "qty": 5
        }
    }
}
response = fyers.modify_gtt_order(data=data)
print(response)

--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------
  {
  "code": 1102,
  "message": "Successfully modified order',
```

```
"s": "ok",
"id": '8102710298291"
}
```


GTT Cancel Order

You can cancel pending orders before they trigger.

Request attributes - For each order

Attribute	Data Type	Description
`id*`	string	Unique identifier for the order to be cancelled, e.g., "25010700000001".

Response attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	Status code indicating the outcome of the cancellation request: - 1103: Order cancellation successful. - Other codes represent specific error conditions.
`message`	string	Descriptive message about the cancellation request, e.g., "Successfully
`id`	string	Unique identifier for the cancelled order, returned if the cancellation is successful.

Request samples

cURL | Python | Node | Web Modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel
```

```
client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, is_async=False, token=acce


data = {"id":'25012400002099'}

response = fyers.cancel_gtt_order(data=data)
print(response)

-------------------------------------------------------------------------------
Sample Success Response
-------------------------------------------------------------------------------
{
  "code": 1103,
  "message": "Successfully cancelled order",
  "s": "ok",
  "id": "25012400002099"
}
```

GTT Order Book

You can fetch all pending GTT Orders.

Response attributes - For each order

Attribute	Data Type	Description
`code`	integer	Status code of the API response: - 200: Request successful. - Other codes represent specific error conditions.
`message`	string	Optional message about the API request (can be empty for successful requests).
`s`	string	Status of the request: "ok" for success, "error" for failure.
`orderBook`	array	List of order details.

Attribute	Data Type	Description
`orderBook[].client_id`	string	Client identifier associated with the order.
`orderBook[].exchange`	integer	Exchange code where the order is placed, e.g., 10 for NSE.
`orderBook[].fy_token`	string	Unique token for the instrument in the order.
`orderBook[].id_fyers`	string	Fyers system-generated unique identifier for the order.
`orderBook[].id`	string	Order ID provided by the system.
`orderBook[].instrument`	integer	Instrument type, e.g., 0 for equity.
`orderBook[].lot_size`	integer	Lot size of the instrument in the order.
`orderBook[].multiplier`	integer	Multiplier for the instrument (e.g., for derivatives).
`orderBook[].ord_status`	integer	Order status code: - • 1: Cancelled - • 2: Traded / Filled - • 3: For future use - • 4: Transit - • 5: Rejected - • 6: Pending
`orderBook[].precision`	integer	Decimal precision for the instrument's price.
`orderBook[].price_limit`	number	Limit price for the first leg of the order.
`orderBook[].price2_limit`	number	Limit price for the second leg (if OCO).
`orderBook[].price_trigger`	number	Trigger price for the first leg of the order.
`orderBook[].price2_trigger`	number	Trigger price for the second leg (if OCO).
`orderBook[].product_type`	string	Product type, e.g., "CNC", "MARGIN", "MTF".
`orderBook[].qty`	integer	Quantity for the first leg of the order.
`orderBook[].qty2`	integer	Quantity for the second leg (if OCO).
`orderBook[].report_type`	string	Status of the order, e.g., "CANCELLED", "PLACED".
`orderBook[].segment`	integer	Segment code, e.g., 10 for equity.

Attribute	Data Type	Description
`orderBook[].symbol`	string	Symbol of the instrument, e.g., "NSE:CHOLAFIN-EQ".
`orderBook[].symbol_desc`	string	Full description of the symbol, e.g., "CHOLAMANDALAM IN & FIN CO".
`orderBook[].symbol_exch`	string	Symbol short code on the exchange, e.g., "CHOLAFIN".
`orderBook[].tick_size`	number	Minimum price increment for the instrument.
`orderBook[].tran_side`	integer	Transaction side: 1 for buy, -1 for sell.
`orderBook[].gtt_oco_ind`	integer	Indicator for the order type: - 1: GTT order. 2: OCO order.
`orderBook[].create_time`	string	Creation timestamp of the order in human-readable format.
`orderBook[].create_time_epoch`	integer	Creation timestamp of the order in epoch seconds.
`orderBook[].oms_msg`	string	Message from the Order Management System (OMS) about the order's status.
`orderBook[].ltp_ch`	number	Last traded price change.
`orderBook[].ltp_chp`	number	Last traded price change percentage.
`orderBook[].ltp`	number	Last traded price of the instrument.

Request samples

cURL | Python | Node | Web Modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn
```

```
response = fyers.gtt_orderbook()
print(response)


--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------
Response structure:
{
  "s": "ok",
  "code": 200,
  "message": "",
  "orderBook": [
    {
      "clientId": "X******",
      "exchange": 10,
      "fy_token": "10100000003045",
      "id_fyers": "c6697c04-d9ab-4a7c-a6f4-b0cc4ca698f6",
      "id": "25012400002074",
      "instrument": 0,
      "lot_size": 1,
      "multiplier": 0,
      "ord_status": 1,
      "precision": 2,
      "price_limit": 1020,
      "price2_limit": 620,
      "price_trigger": 1020,
      "price2_trigger": 620,
      "product_type": "CNC",
      "qty": 5,
      "qty2": 5,
      "report_type": "CANCELLED",
      "segment": 10,
      "symbol": "NSE:SBIN-EQ",
      "symbol_desc": "STATE BANK OF INDIA",
      "symbol_exch": "SBIN",
      "tick_size": 0.05,
      "tran_side": 1,
      "gtt_oco_ind": 2,
      "create_time": "24-Jan-2025 16:13:31",
      "create_time_epoch": 1737715411,
      "oms_msg": "GTT/OCO order cancelled successfully.",
      "ltp_ch": 0,
      "ltp_chp": 0,
      "ltp": 0
    },
    {
```

```
        "clientId": "X******",
        "exchange": 10,
        "fy_token": "10100000003045",
        "id_fyers": "142849a0-d32b-44b5-9108-b7db5ee5e59b",
        "id": "25012400002099",
        "instrument": 0,
        "lot_size": 1,
        "multiplier": 0,
        "ord_status": 1,
        "precision": 2,
        "price_limit": 1000,
        "price2_limit": 600,
        "price_trigger": 1000,
        "price2_trigger": 600,
        "product_type": "MTF",
        "qty": 3,
        "qty2": 3,
        "report_type": "CANCELLED",
        "segment": 10,
        "symbol": "NSE:SBIN-EQ",
        "symbol_desc": "STATE BANK OF INDIA",
        "symbol_exch": "SBIN",
        "tick_size": 0.05,
        "tran_side": -1,
        "gtt_oco_ind": 2,
        "create_time": "24-Jan-2025 16:10:27",
        "create_time_epoch": 1737715227,
        "oms_msg": "GTT/OCO order cancelled successfully.",
        "ltp_ch": 0,
        "ltp_chp": 0,
        "ltp": 0
    }
  ]
}
```


Other Transactions

Modify Orders

This allows the user to modify a pending order. User can provide parameters which needs to be modified. In case a particular parameter has not been provided, the original value will be considered.

Request attributes

Attribute	Data Type	Description
`id*`	string	Mandatory. Eg: 119031547242
`type*`	int	Mandatory. Incase you want to change the order type of the pending order
`limitPrice`	float	Mandatory. Only incase of Limit/ Stoplimit orders
`stopPrice`	float	Mandatory. Only incase of Stop/ Stoplimit orders
`qty`	int	Mandatory. Incase you want to modify the quantity
`disclosedQty`	int	Disclosed quantity (Optional)

Response attributes

Attribute	Data Type	***Description**
`s`	string	ok / error
`code`	int	This is the code to identify specific responses If 201 comes in code it implies that order request has been made but no acknowledgement has been received in this case check orderbook before placing order
`message`	string	Message to clarify the request status. Eg: Successfully modified order
`id`	string	The order number of the modified order Eg: 119031547242

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn


orderId = "8102710298291"
data = {
    "id":orderId,
    "type":1,
    "limitPrice": 61049,
    "qty":1
}

response = fyers.modify_order(data=data)
print(response)

-------------------------------------------------------------------------------
Sample Success Response
-------------------------------------------------------------------------------
    {
    "s": "ok",
    "code": 1101,
    "message": "Successfully modified order', 'id': '8102710298291"
  }
```

Modify Multi Orders

You can modify upto 10 orders simultaneously via the API.
While Modifying Multi orders you need to pass an ARRAY containing the orders request attributes

Sample Request :

```
[{
    "id":orderId,
    "type":1,
    "limitPrice": 61049,
    "qty":1
  },
  {
    "id":orderId,
    "type":1,
    "limitPrice": 61049,
```

```
    "qty":1
  }]
```

Request samples

Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"
# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn


data = [{
  "id": 8102710298291,
  "type": 1,
  "limitPrice": 61049,
  "qty": 1
},
{
  "id": 8102710298292,
  "type": 1,
  "limitPrice": 61049,
  "qty": 1
}]

response = fyers.modify_basket_orders(data=data)
print(response)
```

```
--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------
 {
  "s": "ok",
  "code": 200,
  "message": "",
  "data": [{"statusCode": 200,
            "body": {
                        "s": "ok",
                        "code": 200,
                        "message": "Successfully modified order",
```

```
                    "id": 8102710298291
                },
                "statusDescription": "HTTP OK"
            },
            {
              "statusCode": 200,
              "body": {
                        "s": "ok",
                        "code": 200,
                        "message": "Successfully modified order",
                        "id": 8102710298292
                    },
                    "statusDescription": "HTTP OK"
              }
            ]}

}
```


Cancel Order

Cancel pending orders

Request attributes

Attribute	Data Type	Description
`id*`	string	Mandatory. Eg: 119031547242

Response attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	This is the code to identify specific responses If 201 comes in code it implies that order request has been made but no acknowledgement has been received in this case check orderbook before placing order
`message`	string	Message to clarify the request status. Eg: Successfully canceled order

Attribute	Data Type	Description
`id`	string	The order number of the canceled order. Eg: 119031547242

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```
curl -H "Authorization:app_id:access_token" -H "Content-Type: application/jso
  "id": "52009227353"
}' https://api-t1.fyers.in/api/v3/orders/sync


--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------
{
  "s": "ok",
  "code": 1103,
  "message": "Successfully cancelled order",
  "id": "808058117761"
}
```

Cancel Multi Order

You can cancel upto 10 orders simultaneously via the API.
While cancelling Multi orders you need to pass an ARRAY containing the orders request attributes

Sample Request :

```
[{
  "id":orderId
  },
  {
  "id":orderId
}]
```

Request samples

Python | Node | Web modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn


data =  [{
    "id": '808058117761'
},
{
    "id": '808058117762'
}]

response = fyers.cancel_basket_orders(data=data)
print(response)

 ----------------------------------------------------------------------------
 Sample Success Response
 ----------------------------------------------------------------------------
{
   "s":ok,
   "code": 200,
   "message": "",
   "data": [{"statusCode": 200,
               "body":
                     {
                     "s":ok,
                     "code": 1103,
                     "message": "Successfully cancelled order",
                     "id": "808058117761"
                     },"statusDescription": "HTTP OK"
             },
             {
```

```
                "statusCode": 200,
                "body":
                      {
                        "s":ok,
                        "code": 1103,
                        "message": "Successfully cancelled order",
                        "id": "808058117762"
                      }, "statusDescription": "HTTP OK"
              }
            ]
}
```


Exit Position

This allows the user to either exit all open positions or any specific open position.

Request attributes

Attribute	Data Type	Description
`Id`	string	In case id is not passed, then all the open positions will be closed

Response attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	This is the code to identify specific responses . If 201 comes in code it implies that counter order has been placed but not filled due to low liquidity or any other reason
`message`	string	Message to clarify the request status. Eg:All positions are closed

Sample Request :

```
{
  "id": [positionId1,positionId2]
}
```

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

data = {}

response = fyers.exit_positions(data=data)

print(response)
```

```
 ----------------------------------------------------------------------------
 Sample Success Response
 ----------------------------------------------------------------------------
 {
   "s": "ok",
   "code": 200,
   "message": "The position is closed."
 }
```

Exit Position - By Id

Request attributes

Attribute	Data Type	Description
		Optional

Attribute	Data Type	Description
`id`	string or [string]	Eg: NSE:SBIN-EQ-BO This will only exit the open positions for a particular position id or list of position id provided like ["NSE:SBIN-EQ-INTRADAY","NSE:SBIN-EQ-CNC"]

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

data = {
    "id":"NSE:SBIN-EQ-BO"
}

response = fyers.exit_positions(data=data)
print(response)
```

```
 ------------------------------------------------------------------------------
 Sample Success Response
 ------------------------------------------------------------------------------
 {
   "s": "ok",
   "code": 200,
   "message": "The position is closed."
 }
```

Exit Position - By Segment Side & productType

Request attributes

Attribute	Data Type	Description
`segment*`	[int]	**Mandatory** In the API , to close your position for a specific segment, you need to use the "segment" parameter with one of the following possible values: 10(Capital Market),11(Equity Derivatives),12(Currency Derivatives),20(Commodity Derivatives). Eg: [10,12]
`side*`	[int]	**Mandatory** In the API , to close your position for a specific side, you need to use the "side" parameter with one of the following possible values: 1(Buy side),-1(Sell side). Eg: [1,-1]
`productType*`	[string]	**Mandatory** In the API , to close your position for a specific orderType, you need to use the "productType" parameter with one of the following possible values: "INTRADAY","CNC","CO","BO","MARGIN","ALL". Eg: ["INTRADAY","CNC"]

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "SPXXXXE7-100"
access_token = "eyJ0eXAiOiJK***.eyJpc3MiOiJhcGkuZnllcnM***.IzcuRxg4tnXiULCx3*

fyers = fyersModel.FyersModel(client_id=client_id, token=access_token, log_pa

data = {
    "segment":[10],
    "side":[1,-1],
    "productType":["INTRADAY","CNC"]
   }

response = fyers.exit_positions(data=data)
print(response)
```

```
--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------
{
  "s": "ok",
  "code": 200,
  "message": "The position is closed."
}
```

Pending Order Cancel

This is an added functionality for exit position API. If a user has open positions in particular stocks and also working opposite order of the particular stock, the user can close the working orders and then exit the open positions.

Endpoint: https://api-t1.fyers.in/api/v3/positions

The following should be passed in the body of the DELETE method of positions to cancel the pending orders:

```
{"pending_orders_cancel": 1}
```

If only a single symbol's order and position needs to be cancelled, the position ID should also be sent in the body:

```
{"id": "NSE:SBIN-EQ-INTRADAY","pending_orders_cancel": 1}
```

Request samples

cURL

Copy

```
curl -H "Authorization:app_id:access_token" -H "Content-Type: application/jso

--------------------------------------------------------------------------------
```

Sample Success Response


```
{
 "s": "ok",
 "code": 200,
 "message": "The position is closed."
}
```


Convert Position

This allows the user to convert an open position from one product type to another.

Request attributes

Attribute	Data Type	Description
`symbol*`	string	Mandatory. pass the position id here Eg: NSE:SBIN-EQ-INTRADAY
`overnight*`	int	Mandatory. if the position to be converted is a carry forward position, then send overnight flag as 1. If the position is taken today, irrespective of its product type, then send overnight flag as 0
`positionSide*`	int	Mandatory. 1 => Open long positions -1 => Open short positions View Details
`convertQty*`	int	Mandatory. Quantity to be converted. Has to be in multiples of lot size for derivatives
`convertFrom*`	string	Mandatory. Existing productType (CNC positions cannot be converted) View Details
`convertTo*`	string	Mandatory. The new product type View Details

Notes

1. CNC, CO, BO, and MTF positions cannot be converted.
2. You cannot convert positions to CO, BO,MTF.

Response attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	This is the code to identify specific responses
`message`	string	Message to clarify the request status. Eg: You cannot convert CNC position to INTRADAY / MARGIN.

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel


client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn


data = {
    "symbol":"MCX:SILVERMIC20NOVFUT",
    "positionSide":1,
    "convertQty":1,
    "convertFrom":"INTRADAY",
    "convertTo":"CNC"
}

response = fyers.convert_position(data=data)
print(response)
--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------
    {
  "s": "ok",
  "code": 200,
  "message": "Position Converted Successfully!!",
```

```
  "positionDetails": 1101
}
```


Margin Calculator

Span Margin Calculator

Span margin API will calculate the span margin and exposure margin required for the given stock symbols.

Request Attributes

Attribute	Data Type	Description
`symbol`	string	The symbol in fyers symbology format for which margin is calculated. Example - "NSE:NIFTY2292217000CE"
`qty`	integer	The quantity of the particular stock. The quantity should be in multiples of lot size for derivatives.
`side`	integer	Calculate margin for buy or sell order. 1 => Buy, -1 => Sell
`type`	integer	1 => Limit Order 2 => Market Order 3 => Stop Order (SL-M) 4 => Stoplimit Order (SL-L) More details - View Details
`productType`	string	The product type.The possible values are, 1) CNC 2) INTRADAY 3) MARGIN 4) CO 5) BO 6) MTF More details - View Details
`limitPrice`	float	The limit price to calculate margin. (Keep default as 0.0)
`stopLoss`	float	Provide valid price to calculate margin for CO and BO orders. (Keep default as 0.0)

Request samples

cURL

Copy

```
curl --location --request POST 'https://api.fyers.in/api/v2/span_margin' \ --
        "data": [{
          "symbol": "NSE:BANKNIFTY23NOV44400CE",
          "qty": 50,
          "side": -1,
          "type": 2,
          "productType": "INTRADAY",
          "limitPrice": 0.0,
          "stopLoss": 0.0
        }, {
          "symbol": "NSE:BANKNIFTY23NOVFUT",
          "qty": 50,
          "side": -1,
          "type": 2,
          "productType": "INTRADAY",
          "limitPrice": 0.0,
          "stopLoss": 0.0
          }]
```

Multiorder Margin Calculator

Multiorder margin API will calculate the margin required for the given list of order bodies.

Request Attributes

Attribute	Data Type	Description
`symbol`	string	The symbol in fyers symbology format for which margin is calculated. Example - "NSE:NIFTY2292217000CE"
`qty`	integer	The quantity of the particular stock. The quantity should be in multiples of lot size for derivatives.
`side`	integer	Calculate margin for buy or sell order. 1 => Buy, -1 => Sell
`type`	integer	1 => Limit Order 2 => Market Order 3 => Stop Order (SL-M) 4 => Stoplimit Order (SL-L) More details - View Details

Attribute	Data Type	Description
`productType`	string	The product type.The possible values are, 1) CNC 2) INTRADAY 3) MARGIN 4) CO 5) BO 6) MTF More details - View Details
`limitPrice`	float	The limit price to calculate margin. (Keep default as 0.0)
`stopLoss`	float	Provide valid price to calculate margin for CO and BO orders. (Keep default as 0.0)
`stopPrice`	float	The stop price to calculate margin.(Keep default as 0.0) .(Keep default as 0.0)
`takeProfit`	float	Provide valid take profit price to calculate margin for CO and BO orders.(Keep default as 0.0)

Response attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	This is the code to identify specific responses
`margin_total`	float	Approximate margin required for the order.
`margin_new_order`	float	Approximate Total margin required including existing positions
`margin_avail`	float	Approximate available margin

Request samples

cURL

Copy

```
curl --location --request POST 'https://api-t1.fyers.in/api/v3/multiorder/mar
  "data": [{
    "symbol": "NSE:NIFTY23DECFUT",
    "qty": 50,
    "side": 1,
    "type": 2,
    "productType": "MARGIN",
```

```
  "limitPrice": 0.0,
  "stopLoss": 0.0,
  "stopPrice":0.0,
  "takeProfit":0.0
}, {
  "symbol": "NSE:SBIN-EQ",
  "qty": 50,
  "side": 1,
  "type": 2,
  "productType": "MARGIN",
  "limitPrice": 0.0,
  "stopLoss": 0.0,
  "stopPrice":0.0,
  "takeProfit":0.0
  }]
--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------
{
  "code": 200,
  "message": "",
  "data": {
      "margin_avail": 1999.9,
      "margin_total": 147738.05634886527,
      "margin_new_order": 147738.05634886527
  },
  "s": "ok"
}
```


Broker Config

Market Status

Fetches the current market status of all the exchanges and their segments

Response attributes - For each exchange market segment

Attribute	Data Type	Description
`exchange`	int	The exchange in which the position is taken. View Details
`segment`	int	The segment in which the position is taken. View Details
`market_type`	string	NORMAL, ODD_LOT, CALL_AUCTION2, AUCTION
`status`	string	CLOSE OPEN POSTCLOSE_START POSTCLOSE_CLOSED PREOPEN PREOPEN_CLOSED

Request samples

cURL | python | node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel


client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn


response = fyers.market_status()
print(response)
 ---------------------------------------------------------------------------
 Sample Success Response
 ---------------------------------------------------------------------------

  {
    "code":200,
    "marketStatus":[
        {
          "exchange":10,
          "market_type":"NORMAL",
```

```
      "segment":10,
      "status":"POSTCLOSE_CLOSED"
    },
    {
      "exchange":10,
      "market_type":"NORMAL",
      "segment":11,
      "status":"CLOSED"
    },
    {
      "exchange":10,
      "market_type":"NORMAL",
      "segment":12,
      "status":"CLOSED"
    },
    {
      "exchange":10,
      "market_type":"CALL_AUCTION2",
      "segment":10,
      "status":"CLOSED"
    },
    {
      "exchange":10,
      "market_type":"AUCTION",
      "segment":10,
      "status":"CLOSED"
    },
    {
      "exchange":10,
      "market_type":"ODD_LOT",
      "segment":10,
      "status":"CLOSED"
    },
    {
      "exchange":11,
      "market_type":"NORMAL",
      "segment":20,
      "status":"OPEN"
    },
    {
      "exchange":11,
      "market_type":"SPECIAL",
      "segment":20,
      "status":"CLOSED"
    },
    {
      "exchange":12,
      "market_type":"NORMAL",
```

```
            "segment":10,
            "status":"POSTCLOSE_CLOSED"
          },
          {
            "exchange":12,
            "market_type":"NORMAL",
            "segment":12,
            "status":"POSTCLOSE_CLOSED"
          },
          {
            "exchange":12,
            "market_type":"AUCTION",
            "segment":10,
            "status":"CLOSED"
          },
          {
            "exchange":12,
            "market_type":"NORMAL",
            "segment":11,
            "status":"POSTCLOSE_CLOSED"
          }
      ],
      "message":"",
      "s":"ok"
    }
```


Symbol Master

You can get all the latest symbols of all the exchanges from the symbol master files

- **NSE – Currency Derivatives:** https://public.fyers.in/sym_details/NSE_CD.csv
- **NSE – Equity Derivatives:** https://public.fyers.in/sym_details/NSE_FO.csv
- **NSE – Commodity:** https://public.fyers.in/sym_details/NSE_COM.csv
- **NSE – Capital Market:** https://public.fyers.in/sym_details/NSE_CM.csv
- **BSE – Capital Market:** https://public.fyers.in/sym_details/BSE_CM.csv
- **BSE - Equity Derivatives:** https://public.fyers.in/sym_details/BSE_FO.csv
- **MCX - Commodity:**

https://public.fyers.in/sym_details/MCX_COM.csv

File Headers

Attribute	Data Type	Description
`Fytoken`	string	Unique token for each symbol View Details
`Symbol Details`	string	Name of the symbol
`Exchange Instrument type`	int	Exchange instrument type View Details
`Minimum lot size`	int	Minimum qty multiplier
`Tick size`	float	Minimum price multiplier
`ISIN`	string	Unique ISIN provided by exchange for each symbol
`Trading Session`	string	Trading session provided in IST
`Last update date`	date	Date of last update
`Expiry date`	string	Date of expiry for a symbol.Applicable only for derivative contracts
`Symbol ticker`	string	Unique string to identify the symbol
`Exchange`	int	Exchange mapping View Details
`Segment`	int	Segment of the symbol View Details
`Scrip code`	int	Token of the Exchange
`Underlying symbol`	string	name of underlying symbol
`Underlying scrip code`	int	Scrip code of underlying symbol
`Strike price`	float	Strike price
`Option type`	string	CE/PE - For options XX - For other segments
`Underlying FyToken`	string	Unique token for the underlying symbol
`Reserved column`	string	Reserved for future, kindly ignore
`Reserved column`	int	Reserved for future, kindly ignore
`Reserved column`	float	Reserved for future, kindly ignore

Attribute	Data Type	Description

Symbol Master Json

You can get all the latest symbols of all the exchanges from the symbol master json files

- **NSE – Currency Derivatives:** https://public.fyers.in/sym_details/NSE_CD_sym_master.json
- **NSE – Equity Derivatives:** https://public.fyers.in/sym_details/NSE_FO_sym_master.json
- **NSE – Commodity:** https://public.fyers.in/sym_details/NSE_COM_sym_master.json
- **NSE – Capital Market:** https://public.fyers.in/sym_details/NSE_CM_sym_master.json
- **BSE – Capital Market:** https://public.fyers.in/sym_details/BSE_CM_sym_master.json
- **BSE - Equity Derivatives:** https://public.fyers.in/sym_details/BSE_FO_sym_master.json
- **MCX - Commodity:** https://public.fyers.in/sym_details/MCX_COM_sym_master.json

File Format

Key will be the symbol ticker and value will hold the below json object which has symbol master details for that particular symbol ticker.

Key	Data Type Of Values	Description
`fyToken`	string	Unique token for the security View Details
`isin`	string	ISIN code for the security
`exSymbol`	string	Exchange symbol of the security
`symDetails`	string	Full name of the security
`symTicker`	string	Unique string to identify the security

Key	Data Type Of Values	Description
`exchange`	int	Exchange mapping View Details
`segment`	int	Segment of the security View Details
`exSymName`	string	Symbol name of the security
`exToken`	int	Exchange token
`exSeries`	string	Series of the security. This can be used only for CM segments
`optType`	string	CE/PE - For options XX - For others
`underSym`	string	name of underlying symbol
`underFyTok`	string	Unique token for the underlying symbol
`exInstType`	int	Exchange instrument type View Details
`minLotSize`	int	Minimum qty multiplier
`tickSize`	float	Minimum price multiplier
`tradingSession`	string	Trading session provided in IST
`lastUpdate`	string	Date of last update in YYYY-MM-DD format
`expiryDate`	string	Date of expiry for a symbol in timestamp.Applicable only for derivative contracts
`strikePrice`	float	Strike price
`qtyFreeze`	string	Freeze qty Empty string in case value is not applicable
`tradeStatus`	int	Flag to check if security is allowed to trade. Expected values - 0/1 0 = Tradable. 1 = Non-tradable
`currencyCode`	string	Currency code
`upperPrice`	float	Upper circut price
`lowerPrice`	float	Lower circut price
`faceValue`	float	Face value
`qtyMultiplier`	float	Quantity multipler

Key	Data Type Of Values	Description
`previousClose`	float	Previous close price
`previousOi`	float	Previous OI value
`asmGsmVal`	string	Surveillance Indicator message
`exchangeName`	string	Exchange Name NSE/BSE/MCX
`symbolDesc`	string	Full name of the security
`originalExpDate`	string	Kindly ignore this field
`is_mtf_tradable`	int	0: Not MTF tradable. 1: MTF tradable.
`mtf_margin`	float	Indicates the margin multiplier available for MTF transactions. For example, a value of 2.9 means you receive 2.9x margin on the stock or security.
`stream`	string	Stream Group

EDIS

Electronic Delivery Instruction Slip (eDIS) allows you to sell shares if your Power of Attorney (POA) is not submitted or DDPI is not activated.

Please note: You can only sell the authorized stocks that you are holding in your Demat account. You can **activate DDPI** to ensure a smooth and uninterrupted trading experience.

TPIN Generation

TPIN is an authorization code generated by CDSL/NSDL respectively, using which the customer validates/authorises the transaction.

Request samples

cURL

Copy

```
curl --location --request GET 'https://api.fyers.in/api/v2/tpin'
--header 'Authorization: app_id:access_token'

--------------------------------------------------------------------------------
Sample Success Response
--------------------------------------------------------------------------------

{"s": "ok", "code": 200, "message": "Successfully sent request for BO Tpin ge
```

Details

This API provides information about holding authorizations that have been successfully completed.

Request samples

cURL

Copy

```
curl --location --request GET 'https://api.fyers.in/api/v2/details' \
 --header 'Authorization: app_id:access_token'

  --------------------------------------------------------------------------------
 Sample Success Response
 --------------------------------------------------------------------------------

 {"s": "ok", "code": 200, "message": "", "data": [{"clientId": "DXXXX4", "isi
```

Index

This Api will provide you with the CDSL page to login where you can submit your Holdings information and accordingly you can provide the same to exchange to Sell your holdings.

Request samples

cURL

Copy

```
curl --location --request POST 'https://api.fyers.in/api/v2/index' --header '
 --data-raw '{"recordLst": [{"isin_code": "INE114A01011","qty": "1","symbol":

  ---------------------------------------------------------------------------
 Sample Success Response
 ---------------------------------------------------------------------------

  {"s": "ok", "code": 200, "message": "", "data": "<table width=\"100%\"><tr>
```

Inquiry

This Api is used to get the information/status of the provided transaction Id for the respective holdings you have on your end.
Note: Transaction ID to be base64 encoded in payload. please refer to the online tool for conversion.

Request samples

cURL

Copy

```
curl --location --request POST 'https://api.fyers.in/api/v2/inquiry'
 --header 'Authorization: app_id:access_Token' --header 'Content-Type: applic

 ------------------------------------------------------------------------------
Sample Success Response
------------------------------------------------------------------------------

 {"s": "ok", "code": 200, "message": "", "data": {"FAILED_CNT": 0, "SUCEESS_
```

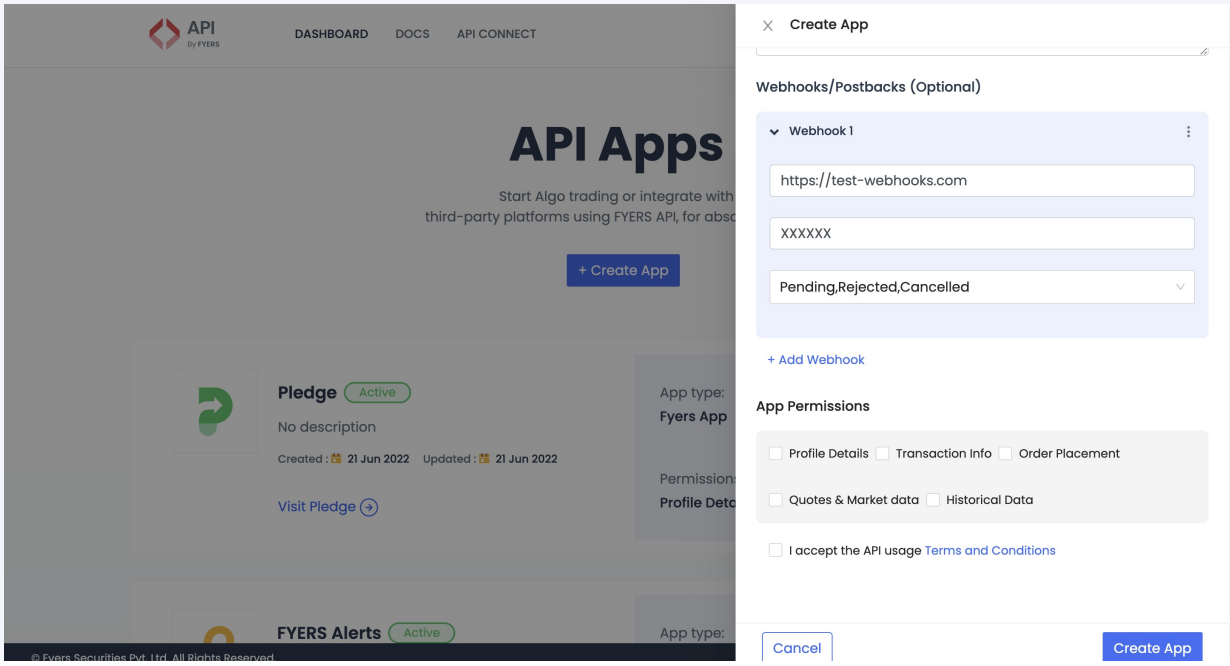
Postback (Webhooks)

The Postback API sends a POST request with a JSON payload to the registered postback_url of your app when an order's status changes. This enables you to get orders updates reliably, irrespective of when they happen (Pending, Cancel, Rejected, Traded).

1. Create web hooks

To Create Postback, you need to create an App by:

1. Login to **API Dashboard**.
2. Once you have logged into the Dashboard, you will see a block where you need to update your webhook URL, webhook secret, and the webhook preference (Cancel, Rejected, Pending, Traded). Here you can add multiple webhooks by clicking on the "Add webhook" button.



3. After entering the required details, click on the "Create App" button by accepting the terms and conditions.

2. Active App

After successful creation of the App, you need to activate the App by following the Authentication & Login mechanism. After successfully logging in via the App, you will be able to get the order data over the webhook URL.

3. Response

The JSON payload is posted as a raw HTTP POST body. You will have to read the raw body.

Attribute	Data Type	Description
id	string	The unique order ID assigned for each order
exchOrdId	string	The order ID provided by the exchange
symbol	string	The symbol for which the order is placed
qty	int	The original order quantity
remainingQuantity	int	The remaining quantity
filledQty	int	The filled quantity after partial trades
status	int	1 => Canceled, 2 => Traded / Filled, 3 => (Not used currently), 4 => Transit, 5 => Rejected, 6 => Pending, 7 => Expired.
message	string	The error messages are shown here
segment	int	10 (Equity), 11 (F&O), 12 (Currency), 20 (Commodity). **View Details**
limitPrice	float	The limit price for the order
stopPrice	float	The stop price for the order
productType	string	The product type
type	int	1 => Limit Order, 2 => Market Order, 3 => Stop Order (SL-M), 4 => Stop Limit Order (SL-L). **View Details**

Attribute	Data Type	Description
side	int	Disclosed quantity: 1 => Buy, -1 => Sell. **View Details**
disclosedQty	int	Disclosed quantity
dqQtyRem	int	Remaining disclosed quantity
orderValidity	string	DAY, IOC
orderDateTime	string	The datetime object here will be in epoch timestamp
tradedPrice	float	The average traded price for the order
source	string	Source from where the order was placed. **View Details**
fytoken	string	Fytoken is a unique identifier for every symbol. **View Details**
offlineOrder	boolean	False => When the market is open, True => When placing AMO order
pan	string	PAN of the client
clientId	string	The client ID of the Fyers user
exchange	int	The exchange in which the order is placed. **View Details**
instrument	string	Exchange instrument type. **View Details**

Response Output :


```
{
  "orderDateTime": "18-Jul-2023 11:44:29",
  "id": "23071800238607",
  "exchOrdId": "2500000061319029",
  "side": -1,
  "segment": 11,
  "instrument": 15,
  "productType": "MARGIN",
  "status": 2,
  "qty": 5400,
  "remainingQuantity": 0,
  "filledQty": 5400,
  "limitPrice": 2.15,
  "stopPrice": 0,
  "type": 2,
  "discloseQty": 0,
  "dqQtyRem": 0,
  "orderValidity": "DAY",
  "source": "M",
  "fyToken": "101123072754619",
  "offlineOrder": false,
  "message": "Completed",
  "orderNumStatus": "23071800238607:2",
  "tradedPrice": 2.15,
  "exchange": 10,
  "pan": "",
  "clientId": "xxxx",
  "symbol": "NSE:ABCAPITAL23JUL190CE"
}
```

4. Blacklisting

The webhook/postback URL provided by the user can be blacklisted if the response status is not 200 from your web server or if the post request to your PostbackURL fails. These URLs are blacklisted for 30 minutes, and during that time, no webhooks are sent to the blacklisted URL. Additionally, the URL will be blacklisted after continuous failures, typically after three retries.

Data Api

History

The historical API provides archived data (up to date) for the symbols. across various exchanges within the given range. A historical record is presented in the form of a candle and the data is available in different resolutions like - minute, 10 minutes, 60 minutes...240 minutes and daily.

To Handle partial Candle

To receive completed candle data, it is important to send a timestamp that comes before the current minute. If you send a timestamp for the current minute, you will receive partial data because the minute is not yet finished. Therefore, it is recommended to always use a "range_to" timestamp of the previous minute to ensure that you receive the completed candle data.

Example:
Current Time(seconds can be 1-59): 12:10:20 PM

Input for history will be:

- range_from: 12:08:00 PM
- range_to: Current Time - 1 minute = 12:09:20 PM

So you will get 2 candles - 12:08 PM and 12:09 PM candles. This example is for 1-minute candles; for other resolutions, you have to subtract the resolution time from "range_to" to get completed candles only.

Limits for History

- Unlimited number of stocks history data can be downloaded in a day.

- Up to 100 days of data per request for 1, 2, 3, 5, 10, 15, 20, 30, 45, 60, 120, 180, and 240 minutes resolution.
- For 1D resolutions up to 366 days of data per request for 1D (1 day) resolutions.
- For Seconds Charts the history will be available only for 30-Trading Days

Request Attribute

Attribute	Data Type	Description
`symbol*`	string	Mandatory. Eg: NSE:SBIN-EQ
`resolution*`	string	The candle resolution. Possible values are: Day : “D” or “1D” 5 seconds : “5S” 10 seconds : “10S” 15 seconds : “15S” 30 seconds : “30S” 45 seconds : “45S” 1 minute : “1” 2 minute : “2" 3 minute : "3" 5 minute : "5" 10 minute : "10" 15 minute : "15" 20 minute : "20" 30 minute : "30" 60 minute : "60" 120 minute : "120" 240 minute : "240"
`date_format*`	int	date_format is a boolean flag. 0 to enter the epoch value. Eg:670073472. 1 to enter the date format as yyyy-mm-dd.
`range_from*`	string	Indicating the start date of records. Accepts epoch value if date_format flag is set to 0. Eg: range_from: 670073472 Accepts yyyy-mm-dd format if date_format flag is set to 1. Eg: 2021-01-01
`range_to*`	string	Indicating the end date of records. Accepts epoch value if date_format flag is set to 0. Eg: range_to: 1622028732 Accepts yyyy-mm-dd format if date_format flag is set to 1. Eg:2021-03-01
`cont_flag*`	int	set cont flag 1 for continues data and future options.
`oi_flag`	int	set flag to "1" enable oi as a part of candle.

Response Attribute

Attribute	Data Type	Description
`s`	string	ok / error
`Candels`	array	Candles data containing array of following data for particular time stamp: 1.Current epoch time 2. Open Value 3.Highest Value 4.Lowest Value 5.Close Value 6.Total traded quantity (volume)

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, is_async=False, token=acce

data = {
    "symbol":"NSE:SBIN-EQ",
    "resolution":"D",
    "date_format":"0",
    "range_from":"1690895316",
    "range_to":"1691068173",
    "cont_flag":"1"
}

response = fyers.history(data=data)
print(response)



 ------------------------------------------------------------------------
Sample Success Response
```

```
{
  "candles":[
      [
        1690934400,
        609.85,
        610.5,
        594.1,
        598.45,
        14977497
      ],
      [
        1691020800,
        598.7,
        600.85,
        585,
        590.5,
        27774877
      ]
  ],
  "code":200,
  "message":"",
  "s":"ok"
}
```


Quotes

The Quotes API retrieves the full market quotes for one or more symbols provided by the user.

Request attributes

Attribute	Data Type	*Description
`symbols*`	string	Maximum symbol limit is 50. Eg: NSE:SBIN-EQ.

Response attributes

Attribute	Data Type	Description
`s`	string	ok / error

Attribute	Data Type	Description
`ch`	float	Change value
`chp`	float	Percentage of change between the current value and the previous day's market close
`lp`	float	Last traded price
`spread`	float	Difference between lowest asking and highest bidding price
`ask`	float	Asking price for the symbol
`bid`	float	Bidding price for the symbol
`open_price`	float	Price at market opening time
`high_price`	float	Highest price for the day
`low_price`	float	Lowest price for the day
`prev_close_price`	float	Previous closing price
`atp`	float	Average traded price
`volume`	int	Volume traded
`short_name`	string	Short name for the symbol Eg: “SBIN-EQ”
`exchange`	string	Name of the exchange. Eg: “NSE” or “BSE”
`description`	string	Description of the symbol
`original_name`	string	Original name of the symbol name provided by the use
`symbol`	string	Symbol name provided by the user
`fyToken`	string	Unique token for each symbol
`tt`	int	Today’s time
`cmd`	dict	---Deprecated---

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel

client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"
# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

data = {
    "symbols":"NSE:SBIN-EQ,NSE:IDEA-EQ"
}

response = fyers.quotes(data=data)
print(response)
```

Sample Success Response

```
 {
   "code":200,
   "d":[
       {
         "n":"NSE:SBIN-EQ",
         "s":"ok",
         "v":{
             "ask":0,
             "bid":590.5,
             "ch":-7.95,
             "chp":-1.33,
             "description":"NSE:SBIN-EQ",
             "exchange":"NSE",
             "fyToken":"10100000003045",
             "high_price":600.85,
             "low_price":585,
             "lp":590.5,
             "open_price":598.7,
             "original_name":"NSE:SBIN-EQ",
             "prev_close_price":598.45,
             "atp": 428.07
             "short_name":"SBIN-EQ",
             "spread":-590.5,
             "symbol":"NSE:SBIN-EQ",
             "tt":"1691020800",
             "volume":27774877
         }
       },
       {
         "n":"NSE:IDEA-EQ",
         "s":"ok",
         "v":{
```

```
                "ask":7.85,
                "bid":0,
                "ch":-0.05,
                "chp":-0.63,
                "description":"NSE:IDEA-EQ",
                "exchange":"NSE",
                "fyToken":"101000000014366",
                "high_price":8.05,
                "low_price":7.8,
                "lp":7.85,
                "open_price":7.9,
                "original_name":"NSE:IDEA-EQ",
                "prev_close_price":7.9,
                "atp": 7.5
                "short_name":"IDEA-EQ",
                "spread":7.85,
                "symbol":"NSE:IDEA-EQ",
                "tt":"1691020800",
                "volume":78116800
            }
        }
    ],
    "message":"",
    "s":"ok"
}
```


Market Depth

The Market Depth API returns the complete market data of the symbol provided. It will include the quantity, OHLC values, and Open Interest fields, and bid/ask prices.

Request attributes

Attribute	Data Type	Description
`symbol*`	string	Mandatory. Eg: NSE:SBIN-EQ
`ohlcv_flag*`	int	Set the ohlcv_flag to 1 to get open, high, low, closing and volume quantity'

Response attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	This is the code to identify specific responses
`totalbuyqty`	int	Total buying quantity
`totalsellqty`	int	Total selling quantity
`bids`	array	Bidding price along with volume and total number of orders
`ask`	array	Offer price with volume and total number of orders
`o`	float	Price at market opening time
`h`	float	Highest price for the day
`i`	float	Lowest price for the day
`c`	float	Price at the of market closing
`chp`	float	Percentage of change between the current value and the previous day's market close
`tick_Size`	float	Minimum price multiplier
`ch`	float	Change value
`ltq`	int	Last traded quantity
`ltt`	int	Last traded time
`ltp`	float	Last traded price
`v`	int	Volume traded
`atp`	float	Average traded price
`lower_ckt`	float	Lower circuit price`
`upper_ckt`	float	upper circuit price
`expiry`	string	Expiry date
`oi`	float	Open interest
`oiflag`	bool	Boolean flag for OI data, true or false
`pdoi`	int	previous day open interest
`oipercent`	float	Change in open Interest percentage

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3 import fyersModel


client_id = "XC4XXXXM-100"
access_token = "eyJ0eXXXXXXXX2c5-Y3RgS8wR14g"

# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn

data = {
    "symbol":"NSE:SBIN-EQ",
    "ohlcv_flag":"1"
}

response = fyers.depth(data=data)
print(response)
```

```
 ------------------------------------------------------------------------------
 Sample Success Response
 ------------------------------------------------------------------------------
 {
 "s": "ok",
 "d": {
     "NSE:SBIN-EQ": {
         "totalbuyqty": 2396063,
         "totalsellqty": 4990001,
         "bids": [
             {
                 "price": 427.25,
                 "volume": 4738,
                 "ord": 5
             },
             {
                 "price": 427.2,
                 "volume": 2631,
                 "ord": 9
             },
             {
```

```
                "price": 427.15,
                "volume": 6366,
                "ord": 19
            },
            {
                "price": 427.1,
                "volume": 6344,
                "ord": 18
            },
            {
                "price": 427.05,
                "volume": 8136,
                "ord": 16
            }
        ],
        "ask": [
            {
                "price": 427.4,
                "volume": 2193,
                "ord": 4
            },
            {
                "price": 427.45,
                "volume": 5406,
                "ord": 19
            },
            {
                "price": 427.5,
                "volume": 15311,
                "ord": 57
            },
            {
                "price": 427.55,
                "volume": 11170,
                "ord": 17
            },
            {
                "price": 427.6,
                "volume": 7272,
                "ord": 25
            }
        ],
        "o": 430.5,
        "h": 433.65,
        "l": 423.6,
        "c": 425.2,
        "chp": 0.48,
        "tick_Size": 0.05,
```

```
            "ch": 2.05,
            "ltq": 20,
            "ltt": 1622184920,
            "ltp": 427.25,
            "v": 39163870,
            "atp": 428.07,
            "lower_ckt": 382.7,
            "upper_ckt": 467.7,
            "expiry": "",
            "oi": 0,
            "oiflag": false,
            "pdoi": 0,
            "oipercent": 0.0
        }
    },
     "message": ""
     }
```


Option Chain

The Optionchain API provides important information about options trading, specifically focusing on strike prices, which are predetermined prices at which an option contract can be exercised. This API offers data for both Call (CE) and Put (PE) options, along with values for index, IndiaVIX (Volatility Index) and available expiry dates. When using the API, you can specify the number of strike prices you're interested in. For example, if you request a strike count of 2, the API will return data for:

- **At-The-Money (ATM) strike:** The closest strike price to the current market price.
- **Out-of-The-Money (OTM) strikes:** Strike prices higher than the current market price for Calls (CE) and lower than the current market price for Puts (PE). The API will return data for 2 CE and 2 PE OTM strikes.
- **In-The-Money (ITM) strikes:** Strike prices lower than the current market price for Calls (CE) and higher than the current market price for Puts (PE). The API will return data for 2 CE and 2 PE ITM strikes.

.

Request attributes

Attribute	Data Type	Description
`symbol*`	string	Mandatory. Eg: NSE:SBIN-EQ
`strikecount`	int	Options strike count for symbol(MAX = 50)

Attribute	Data Type	Description
`timestamp`	string	Options chain data at timestamp'

Response attributes

Attribute	Data Type	Description
`s`	string	ok / error
`code`	int	This is the code to identify specific responses
`callOi`	int	Total open interest for call options
`putOi`	int	Total open interest for put options
`date`	string	Expiry date in DD-MM-YYYY
`expiry`	string	Expiry timestamp
`ask`	float	Ask price
`bid`	float	Bid price
`description`	string	Description of the stock
`ex_symbol`	string	Exchange Symbol
`exchange`	int	The exchange in which order is placed. View Details
`fytoken`	string	Fytoken is a unique identifier for every symbol. View Details
`ltp`	float	LTP is the price from which the next sale of the stocks happens
`ltpch`	float	Change in last traded price
`ltpchp`	float	Percentage change in last traded price
`option_type`	string	Type of option (if applicable)
`strike_price`	int	Strike price (if applicable)
`symbol`	string	Symbol
`fp`	float	Future price
`fpch`	float	Change in future price
`fpchp`	float	Percentage change in future price
`oi`	int	Open interest

Attribute	Data Type	Description
`oich`	int	Change in Open interest
`oichp`	float	Percentage change in Open interest
`prev_oi`	int	Previous Open interest
`volume`	int	Volume traded

Request samples

cURL | Python | Node | Web modular API | C# | Java

Copy

```
from fyers_apiv3 import fyersModel
client_id = "YYYYYYY-100"
access_token = "XXXXXXXXXXXXX"
# Initialize the FyersModel instance with your client_id, access_token, and e
fyers = fyersModel.FyersModel(client_id=client_id, token=access_token,is_asyn
data = {
    "symbol":"NSE:TCS-EQ",
    "strikecount":1,
    "timestamp": ""
}
response = fyers.optionchain(data=data);
print(response)
 ------------------------------------------------------------------------------
 Sample Success Response
 ------------------------------------------------------------------------------
 {
  "code": 200,
  "data": {
    "callOi": 10038175,
    "expiryData": [
      {
        "date": "25-04-2024",
        "expiry": "1714039200"
      },
      {
        "date": "30-05-2024",
        "expiry": "1717063200"
      },
```

```
      {
        "date": "27-06-2024",
        "expiry": "1719482400"
      }
    ],
    "indiavixData": {
      "ask": 0,
      "bid": 0,
      "description": "INDIAVIX-INDEX",
      "ex_symbol": "INDIAVIX",
      "exchange": "NSE",
      "fyToken": "101000000026017",
      "ltp": 10.55,
      "ltpch": -2.15,
      "ltpchp": -16.93,
      "option_type": "",
      "strike_price": -1,
      "symbol": "NSE:INDIAVIX-INDEX"
    },
    "optionsChain": [
      {
        "ask": 3880.15,
        "bid": 3880.05,
        "description": "TATA CONSULTANCY SERV LT",
        "ex_symbol": "TCS",
        "exchange": "NSE",
        "fp": 3876.65,
        "fpch": 14.2,
        "fpchp": 0.37,
        "fyToken": "101000000011536",
        "ltp": 3880.15,
        "ltpch": 15.55,
        "ltpchp": 0.4,
        "option_type": "",
        "strike_price": -1,
        "symbol": "NSE:TCS-EQ"
      },
      {
        "ask": 34.9,
        "bid": 34.35,
        "fyToken": "1011240425139431",
        "ltp": 34.8,
        "ltpch": 2.7,
        "ltpchp": 8.41,
        "oi": 99575,
        "oich": -3325,
        "oichp": -3.23,
        "option_type": "CE",
```

```
      "prev_oi": 102900,
      "strike_price": 3860,
      "symbol": "NSE:TCS24APR3860CE",
      "volume": 202650
    },
    {
      "ask": 19.3,
      "bid": 19,
      "fyToken": "1011240425139432",
      "ltp": 19.05,
      "ltpch": -12.4,
      "ltpchp": -39.43,
      "oi": 159075,
      "oich": 28525,
      "oichp": 21.85,
      "option_type": "PE",
      "prev_oi": 130550,
      "strike_price": 3860,
      "symbol": "NSE:TCS24APR3860PE",
      "volume": 304150
    },
    {
      "ask": 24.85,
      "bid": 24.55,
      "fyToken": "1011240425133432",
      "ltp": 24.95,
      "ltpch": -0.55,
      "ltpchp": -2.16,
      "oi": 165900,
      "oich": 9975,
      "oichp": 6.4,
      "option_type": "CE",
      "prev_oi": 155925,
      "strike_price": 3880,
      "symbol": "NSE:TCS24APR3880CE",
      "volume": 543025
    },
    {
      "ask": 29.35,
      "bid": 28.8,
      "fyToken": "1011240425133433",
      "ltp": 29.2,
      "ltpch": -14.1,
      "ltpchp": -32.56,
      "oi": 98175,
      "oich": 28350,
      "oichp": 40.6,
      "option_type": "PE",
```

```
        "prev_oi": 69825,
        "strike_price": 3880,
        "symbol": "NSE:TCS24APR3880PE",
        "volume": 199500
      },
      {
        "ask": 17.8,
        "bid": 17.6,
        "fyToken": "1011240425139433",
        "ltp": 17.75,
        "ltpch": -1.4,
        "ltpchp": -7.31,
        "oi": 631050,
        "oich": 23275,
        "oichp": 3.83,
        "option_type": "CE",
        "prev_oi": 607775,
        "strike_price": 3900,
        "symbol": "NSE:TCS24APR3900CE",
        "volume": 981925
      },
      {
        "ask": 42.45,
        "bid": 41.85,
        "fyToken": "1011240425139434",
        "ltp": 41.75,
        "ltpch": -14.65,
        "ltpchp": -25.98,
        "oi": 338100,
        "oich": -9975,
        "oichp": -2.87,
        "option_type": "PE",
        "prev_oi": 348075,
        "strike_price": 3900,
        "symbol": "NSE:TCS24APR3900PE",
        "volume": 129325
      }
    ],
    "putOi": 3875200
  },
  "message": "",
  "s": "ok"
}
```

Web Socket

Introduction

The WebSocket provides a robust method for accessing real-time data or order updates seamlessly and with low latency. It enables developers and users to establish a persistent, bidirectional connection with the server, allowing them to receive continuous updates, such as symbol updates, depth updates or orderupdate. To enhance your experience with our WebSocket, here are some helpful tips and best practices

1. **Subscription Limit:** You have the flexibility to subscribe up to **5000** data subscriptions simultaneously via WebSocket with latest SDK versions, please refer Change Log. Staying within this limit ensures smooth subscription management without errors.
2. **Single Instance:** Keep in mind that you can create only one WebSocket connection instance at a time. This approach ensures stability and prevents issues that might arise from multiple concurrent connections.
3. **Efficient Thread Management:** WebSocket operates on a dedicated thread, allowing it to run independently of your main application thread. This design guarantees that your primary tasks can continue without interruptions from WebSocket operations.
4. **Customizable Callback Functions:** Tailor your application's behavior using callback functions provided by the WebSocket. These functions empower you to define specific actions for events like data updates or error occurrences.
5. **Auto-Reconnect:** Enjoy uninterrupted connectivity by enabling automatic reconnection in case of disconnection. Simply set the reconnect parameter to true during WebSocket initialization, ensuring your application can recover without manual intervention.You can set max reconnection count upto 50.
6. **Logging to File:** If you want to log data to a file, you can set the write_to_file parameter to true. This feature allows you to efficiently save received data to a log file for analysis or archival purposes. The write_to_file function will only work without callback functions.
7. **Reconnect Retry:** If you want to define dynamic retry count(max 50), you can set the reconnect_retry parameter to int value of number of times you want it to try reconnection.(In case of node `fyersdata.autoreconnect(trycount)`)
8. **Disable Logging(node JS):** In case you want to disable logging use disable logging flag to disable logging sample format:
   `new FyersSocket("token","logpath",true/*flag to enable disable logging*/)`

Request samples

Python | Node | C# | Java

```
Copy

from fyers_apiv3.FyersWebsocket import data_ws

fyers = data_ws.FyersDataSocket(
    access_token=access_token,       # Access token in the format "appid:acce
    log_path="",                     # Path to save logs. Leave empty to auto
    litemode=False,                  # Lite mode disabled. Set to True if you
    write_to_file=False,              # Save response in a log file instead o
    reconnect=True,                  # Enable auto-reconnection to WebSocket
    on_connect=onopen,               # Callback function to subscribe to data
    on_close=onclose,                # Callback function to handle WebSocket
    on_error=onerror,                # Callback function to handle WebSocket
    on_message=onmessage,            # Callback function to handle incoming m
    reconnect_retry=10               # Number of times reconnection will be a
)
```

General Socket (orders)

The WebSocket API for receiving order, position, trade, price alerts, and EDIS updates is a real-time communication protocol designed to provide seamless access to various critical elements of a trading and EDIS system. This API allows traders and developers to establish a persistent, bidirectional connection with the server, enabling them to receive real-time updates on their orders, current positions, executed trades, price alerts, and EDIS status.

Response attributes - For order updates

Attribute	Data Type	Description
`id`	string	The unique order id assigned for each order
`exchOrdId`	string	The order id provided by the exchange
`symbol`	string	The symbol for which order is placed
`qty`	int	The original order qty
`remainingQuantity`	int	The remaining qty
`filledQty`	int	The filled qty after partial trades
		1 => Canceled 2 => Traded / Filled 3 => (Not used currently)

Attribute	Data Type	Description
`status`	int	4 => Transit 5 => Rejected 6 => Pending 7 => Expired View Details
`message`	string	The error messages are shown here
`segment`	int	10 => E (Equity) 11 => D (F&O) 12 => C (Currency) 20 => M (Commodity) View Details
`limitPrice`	float	The limit price for the order
`stopPrice`	float	The limit price for the order
`productType`	string	The product type
`type`	int	1 => Limit Order 2 => Market Order 3 => Stop Order (SL-M) 4 => Stoplimit Order (SL-L)
`side`	int	1 => Buy -1 => Sell View Details
`orderValidity`	string	DAY IOC
`orderDateTime`	string	The order time as per DD-MMM-YYYY hh:mm:ss in IST
`parentId`	string	The parent order id will be provided only for applicable orders.. Eg: BO Leg 2 & 3 and CO Leg 2
`tradedPrice`	float	The average traded price for the order
`source`	string	Source from where the order was placed. View Details
`fytoken`	string	Fytoken is a unique identifier for every symbol. View Details
		False => When market is open

Attribute	Data Type	Description
`offlineOrder`	boolean	True => When placing AMO order
`pan`	string	PAN of the client
`clientId`	string	The client id of the fyers user
`exchange`	int	The exchange in which order is placed View Details
`instrument`	int	Exchange instrument type View Details

Request samples

Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3.FyersWebsocket import order_ws

def onOrder(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("Order Response:", message)

def onerror(message):
    """
    Callback function to handle WebSocket errors.

    Parameters:
        message (dict): The error message received from the WebSocket.

    """
    print("Error:", message)


def onclose(message):
```

```
    """
    Callback function to handle WebSocket connection close events.
    """
    print("Connection closed:", message)


def onopen():
    """
    Callback function to subscribe to data type and symbols upon WebSocket co

    """
    # Specify the data type and symbols you want to subscribe to
    data_type = "OnOrders"
    # data_type = "OnTrades"
    # data_type = "OnPositions"
    # data_type = "OnGeneral"
    # data_type = "OnOrders,OnTrades,OnPositions,OnGeneral"

    fyers.subscribe(data_type=data_type)

    # Keep the socket running to receive real-time data
    fyers.keep_running()


# Replace the sample access token with your actual access token obtained from
access_token = "Xxxxx67IM-100:eyJxxxxxxIUzI1NiJ9.eyJpc3MiOiJhcGkuZnllcnMuaW4i

# Create a FyersDataSocket instance with the provided parameters
fyers = order_ws.FyersOrderSocket(
    access_token=access_token,  # Your access token for authenticating with t
    write_to_file=False,        # A boolean flag indicating whether to write
    log_path="",                # The path to the log file if write_to_file i
    on_connect=onopen,          # Callback function to be executed upon succe
    on_close=onclose,           # Callback function to be executed when the W
    on_error=onerror,           # Callback function to handle any WebSocket e
    on_orders=onOrder,          # Callback function to handle order-related e
)


# Establish a connection to the Fyers WebSocket
fyers.connect()


------------------------------------------------------------------------------
Sample Success Response
------------------------------------------------------------------------------

{
```

```
  "s":"ok",
  "orders":{
      "clientId":"XV20986",
      "id":"23080400089344",
      "exchOrdId":"1100000009596016",
      "qty":1,
      "filledQty":1,
      "limitPrice":7.95,
      "type":2,
      "fyToken":"101000000014366",
      "exchange":10,
      "segment":10,
      "symbol":"NSE:IDEA-EQ",
      "instrument":0,
      "offlineOrder":false,
      "orderDateTime":"04-Aug-2023 10:12:58",
      "orderValidity":"DAY",
      "productType":"INTRADAY",
      "side":-1,
      "status":90,
      "source":"W",
      "ex_sym":"IDEA",
      "description":"VODAFONE IDEA LIMITED",
      "orderNumStatus":"23080400089344:2"
  }
}
```


General Socket (trades)

Response attributes - For trade update

Attribute	Data Type	Description
`symbol`	string	Eg: NSE:SBIN-EQ
`id`	string	The unique id to sort the trades
`orderDateTime`	string	The time when the trade occured in “DD-MM-YYYY hh:mm:ss” format in IST
`orderNumber`	string	The order id for which the trade occurred
`tradeNumber`	string	The trade number generated by the exchange

Attribute	Data Type	Description
`tradePrice`	float	The traded price
`tradeValue`	float	The total traded value
`tradedQty`	int	The total traded qty
`side`	int	1 => Buy -1 => Sell View Details
`productType`	string	The product in which the order was placed View Details
`exchangeOrderNo`	string	The order number provided by the exchange
`segment`	int	The segment in which order is placed View Details
`exchange`	int	The exchange in which order is placed View Details
`fyToken`	string	Fytoken is a unique identifier for every symbol.

Request samples

Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3.FyersWebsocket import order_ws


def onTrade(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("Trade Response:", message)
```

```python
def onerror(message):
    """
    Callback function to handle WebSocket errors.

    Parameters:
        message (dict): The error message received from the WebSocket.


    """
    print("Error:", message)


def onclose(message):
    """
    Callback function to handle WebSocket connection close events.
    """
    print("Connection closed:", message)


def onopen():
    """
    Callback function to subscribe to data type and symbols upon WebSocket co

    """
    # Specify the data type and symbols you want to subscribe to
    data_type = "OnTrades"

    # data_type = "OnOrders"
    # data_type = "OnPositions"
    # data_type = "OnGeneral"
    # data_type = "OnOrders,OnTrades,OnPositions,OnGeneral"

    fyers.subscribe(data_type=data_type)

    # Keep the socket running to receive real-time data
    fyers.keep_running()


# Replace the sample access token with your actual access token obtained from
access_token = "Xxxxx67IM-100:eyJxxxxxxIUzI1NiJ9.eyJpc3MiOiJhcGkuZnllcnMuaW4i

# Create a FyersDataSocket instance with the provided parameters
fyers = order_ws.FyersOrderSocket(
    access_token=access_token,  # Your access token for authenticating with t
    write_to_file=False,        # A boolean flag indicating whether to write
    log_path="",                # The path to the log file if write_to_file i
    on_connect=onopen,          # Callback function to be executed upon succe
```

```
    on_close=onclose,           # Callback function to be executed when the W
    on_error=onerror,           # Callback function to handle any WebSocket e
    on_trades=onTrade           # Callback function to handle trade-related e
)


# Establish a connection to the Fyers WebSocket
fyers.connect()


------------------------------------------------------------------------------
Sample Success Response
------------------------------------------------------------------------------
  {
    "s":"ok",
    "trades":{
        "tradeNumber":"23080400089344-21726639",
        "orderNumber":"23080400089344",
        "tradedQty":1,
        "tradePrice":7.95,
        "tradeValue":7.95,
        "productType":"INTRADAY",
        "clientId":"XV20986",
        "exchangeOrderNo":"1100000009596016",
        "orderType":2,
        "side":-1,
        "symbol":"NSE:IDEA-EQ",
        "orderDateTime":"04-08-2023 10:12:58",
        "fyToken":"101000000014366",
        "exchange":10,
        "segment":10
    }
  }
```

General Socket (positions)

Response attributes - For position updates

Attribute	Data Type	Description
`symbol`	string	Eg: NSE:SBIN-EQ

Attribute	Data Type	Description
`id`	string	The unique value for each position
`buyAvg`	float	Average buy price
`buyQty`	int	Total buy qty
`sellAvg`	float	Average sell price
`sellQty`	int	Total sell qty
`netAvg`	float	netAvg
`netQty`	int	Net qty
`side`	int	The side shows whether the position is long / short View Details
`qty`	int	Absolute value of net qty
`productType`	string	The product type of the position View Details
`realized_profit`	float	The realized p&l of the position
`crossCurrency`	string	Y => It is cross currency position N => It is not a cross currency position
`rbiRefRate`	float	Incase of cross currency position, the rbi reference rate will be required to calculate the p&l
`qtyMulti_com`	float	Incase of commodity positions, this multiplier is required for p&l calculation
`segment`	int	The segment in which the position is taken. View Details
`exchange`	int	The exchange in which the position is taken. View Details
`slNo`	int	This is used for sorting of positions
`fytoken`	string	Fytoken is a unique identifier for every symbol. View Details
`cfBuyQty`	int	Carry forward buy quantity
`cfSellQty`	int	Carry forward sell quantity
`dayBuyQty`	int	Day forward buy quantity
`daySellQty`	int	Day forward sell quantity

Attribute	Data Type	Description
`exchange`	int	The exchange in which order is placed View Details
`buyVal`	float	Total buy value of the position.
`sellVal`	float	Total sell value of the position.

Request samples

Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3.FyersWebsocket import order_ws

def onPosition(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("Position Response:", message)


def onerror(message):
    """
    Callback function to handle WebSocket errors.

    Parameters:
        message (dict): The error message received from the WebSocket.


    """
    print("Error:", message)


def onclose(message):
    """
    Callback function to handle WebSocket connection close events.
    """
```

```
    print("Connection closed:", message)


def onopen():
    """
    Callback function to subscribe to data type and symbols upon WebSocket co

    """
    # Specify the data type and symbols you want to subscribe to
    data_type = "OnPositions"

    # data_type = "OnOrders"
    # data_type = "OnTrades"
    # data_type = "OnGeneral"
    # data_type = "OnOrders,OnTrades,OnPositions,OnGeneral"

    fyers.subscribe(data_type=data_type)

    # Keep the socket running to receive real-time data
    fyers.keep_running()


# Replace the sample access token with your actual access token obtained from
access_token = "Xxxxx67IM-100:eyJxxxxxxIUzI1NiJ9.eyJpc3MiOiJhcGkuZnllcnMuaW4i

# Create a FyersDataSocket instance with the provided parameters
fyers = order_ws.FyersOrderSocket(
    access_token=access_token,  # Your access token for authenticating with t
    write_to_file=False,        # A boolean flag indicating whether to write
    log_path="",                # The path to the log file if write_to_file i
    on_connect=onopen,          # Callback function to be executed upon succe
    on_close=onclose,           # Callback function to be executed when the W
    on_error=onerror,           # Callback function to handle any WebSocket e
    on_positions=onPosition,    # Callback function to handle position-relate
)


# Establish a connection to the Fyers WebSocket
fyers.connect()


------------------------------------------------------------------------------
Sample Success Response
------------------------------------------------------------------------------

  {
    "s":"ok",
    "positions":{
```

```
        "symbol":"NSE:IDEA-EQ",
        "id":"NSE:IDEA-EQ-INTRADAY",
        "buyAvg":8,
        "buyQty":1,
        "buyVal":8,
        "sellAvg":7.95,
        "sellQty":1,
        "sellVal":7.95,
        "netAvg":0,
        "netQty":0,
        "side":0,
        "qty":0,
        "productType":"INTRADAY",
        "realized_profit":-0.04999999999999982,
        "rbiRefRate":1,
        "fyToken":"101000000014366",
        "exchange":10,
        "segment":10,
        "dayBuyQty":1,
        "daySellQty":1,
        "cfBuyQty":0,
        "cfSellQty":0,
        "qtyMulti_com":1
    }
}
```


General Socket (general)

The WebSocket API for receiving order, position, trade, price alerts, and EDIS updates is a real-time communication protocol designed to provide seamless access to various critical elements of a trading and EDIS system. This API allows traders and developers to establish a persistent, bidirectional connection with the server, enabling them to receive real-time updates on their orders, current positions, executed trades, price alerts, and EDIS status.

Request samples

Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3.FyersWebsocket import order_ws


def onTrade(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("Trade Response:", message)

def onOrder(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("Order Response:", message)

def onPosition(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("Position Response:", message)

def onGeneral(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("General Response:", message)
def onerror(message):
    """
    Callback function to handle WebSocket errors.

    Parameters:
        message (dict): The error message received from the WebSocket.
```

```
    """
    print("Error:", message)


def onclose(message):
    """
    Callback function to handle WebSocket connection close events.
    """
    print("Connection closed:", message)


def onopen():
    """
    Callback function to subscribe to data type and symbols upon WebSocket co

    """
    # Specify the data type and symbols you want to subscribe to
    # data_type = "OnOrders"
    # data_type = "OnTrades"
    # data_type = "OnPositions"
    # data_type = "OnGeneral"
    data_type = "OnOrders,OnTrades,OnPositions,OnGeneral"

    fyers.subscribe(data_type=data_type)

    # Keep the socket running to receive real-time data
    fyers.keep_running()


# Replace the sample access token with your actual access token obtained from
access_token = "Xxxxx67IM-100:YYYYYYYYYYY"

# Create a FyersDataSocket instance with the provided parameters
fyers = order_ws.FyersOrderSocket(
    access_token=access_token,  # Your access token for authenticating with t
    write_to_file=False,        # A boolean flag indicating whether to write
    log_path="",                # The path to the log file if write_to_file i
    on_connect=onopen,          # Callback function to be executed upon succe
    on_close=onclose,           # Callback function to be executed when the W
    on_error=onerror,           # Callback function to handle any WebSocket e
    on_general=onGeneral,       # Callback function to handle general events
    on_orders=onOrder,          # Callback function to handle order-related e
    on_positions=onPosition,    # Callback function to handle position-relate
    on_trades=onTrade           # Callback function to handle trade-related e
)
```

```
# Establish a connection to the Fyers WebSocket
fyers.connect()

------------------------------------------------------------------------------
Sample Success Response
------------------------------------------------------------------------------

  {
    "s":"ok",
    "orders":{
        "clientId":"XV20986",
        "id":"23080400089344",
        "exchOrdId":"1100000009596016",
        "qty":1,
        "filledQty":1,
        "limitPrice":7.95,
        "type":2,
        "fyToken":"101000000014366",
        "exchange":10,
        "segment":10,
        "symbol":"NSE:IDEA-EQ",
        "instrument":0,
        "offlineOrder":false,
        "orderDateTime":"04-Aug-2023 10:12:58",
        "orderValidity":"DAY",
        "productType":"INTRADAY",
        "side":-1,
        "status":90,
        "source":"W",
        "ex_sym":"IDEA",
        "description":"VODAFONE IDEA LIMITED",
        "orderNumStatus":"23080400089344:2"
    }
  }
```

Market Data Symbol Update

The WebSocket API for receiving stock data is a real-time communication protocol designed to provide seamless and low-latency access to live stock market data. This API allows developers and traders to establish a persistent, bidirectional connection with the server, enabling them to receive continuous updates on stock prices, trading volumes, and other relevant market information.

To quickly get started with the WebSocket API and start receiving real-time stock data, you can explore

our sample scripts and code examples in our GitHub repository:
Data WebSocket Sample Scripts

Response Attributes(Market Data Update)

Attribute	Data Type	Description
`symbol`	string	The symbol in fyers symbology format whose data is
`ltp`	float	Last traded price of security or asset.
`prev_close_price`	float	Previous day's close of security or asset.
`high_price`	float	Current day's high of security or asset.
`low_price`	float	Current day's low of security or asset.
`open_price`	float	Current day's open of security or asset.
`ch`	float	Change in price of security or asset today.
`chp`	float	Change in price in % of security or asset today.
`vol_traded_today`	int	Volume traded for security or asset today.
`last_traded_time`	int	Last traded time of security or asset.
`bid_size`	int	The bid size key represents the quantity or volume of the security or asset that buyers are willing to purchase at a specific bid price level.
`ask_size`	int	The ask size key represents the quantity or volume of the security or asset that sellers are willing to sell at a specific ask price level.
`ask_price`	float	The ask price key represents the lowest price at which sellers are willing to sell a particular security or asset.
`bid_price`	float	The bid price key represents the highest price at which buyers are willing to purchase a particular security or asset.
`last_traded_qty`	int	Last traded quantity of security or asset.
`tot_buy_qty`	int	Total buy quantity of security or asset.
`tot_sell_qty`	int	Total sell quantity of security or asset.
`avg_trade_price`	float	Average trade price of security or asset.
`type`	string	cn - connection message sub - subscribe message if - index data message dp - market depth data message

Attribute	Data Type	Description
		sf - Equity and option data message

Request samples

Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3.FyersWebsocket import data_ws


def onmessage(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("Response:", message)


def onerror(message):
    """
    Callback function to handle WebSocket errors.

    Parameters:
        message (dict): The error message received from the WebSocket.


    """
    print("Error:", message)


def onclose(message):
    """
    Callback function to handle WebSocket connection close events.
    """
    print("Connection closed:", message)


def onopen():
```

```
    """
    Callback function to subscribe to data type and symbols upon WebSocket co

    """
    # Specify the data type and symbols you want to subscribe to
    data_type = "SymbolUpdate"

    # Subscribe to the specified symbols and data type
    symbols = ['NSE:SBIN-EQ', 'NSE:ADANIENT-EQ']
    fyers.subscribe(symbols=symbols, data_type=data_type)

    # Keep the socket running to receive real-time data
    fyers.keep_running()


# Replace the sample access token with your actual access token obtained from
access_token = "XC4XXXXXXM-100:eXXXXXXXXXXXXfZNSBoLo"

# Create a FyersDataSocket instance with the provided parameters
fyers = data_ws.FyersDataSocket(
    access_token=access_token,       # Access token in the format "appid:acce
    log_path="",                     # Path to save logs. Leave empty to auto
    litemode=False,                  # Lite mode disabled. Set to True if you
    write_to_file=False,              # Save response in a log file instead o
    reconnect=True,                  # Enable auto-reconnection to WebSocket
    on_connect=onopen,               # Callback function to subscribe to data
    on_close=onclose,                # Callback function to handle WebSocket
    on_error=onerror,                # Callback function to handle WebSocket
    on_message=onmessage             # Callback function to handle incoming m
)

# Establish a connection to the Fyers WebSocket
fyers.connect()
```

Sample Success Response

```
{
    "ltp":606.4,
    "vol_traded_today":3045212,
    "last_traded_time":1690953622,
    "exch_feed_time":1690953622,
    "bid_size":2081,
    "ask_size":903,
    "bid_price":606.4,
    "ask_price":606.45,
    "last_traded_qty":5,
```

```
    "tot_buy_qty":749960,
    "tot_sell_qty":1092063,
    "avg_trade_price":608.2,
    "low_price":605.85,
    "high_price":610.5,
    "open_price":609.85,
    "prev_close_price":620.2,
    "type":"sf",
    "symbol":"NSE:SBIN-EQ",
    "ch":-13.8,
    "chp":-2.23
  }
```


Market Data Indices Update

The WebSocket API for receiving stock data is a real-time communication protocol designed to provide seamless and low-latency access to live stock market data. This API allows developers and traders to establish a persistent, bidirectional connection with the server, enabling them to receive continuous updates on stock prices, trading volumes, and other relevant market information.

To quickly get started with the WebSocket API and start receiving real-time stock data, you can explore our sample scripts and code examples in our GitHub repository:
Data WebSocket Sample Scripts

Response Attributes(Index Update)

Attribute	Data Type	Description
`symbol`	string	The symbol in fyers symbology format whose data is
`ltp`	float	Last traded price of security or asset.
`prev_close_price`	float	Previous day's close of security or asset.
`high_price`	float	Current day's high of security or asset.
`low_price`	float	Current day's low of security or asset.
`open_price`	float	Current day's open of security or asset.
`ch`	float	Change in price of security or asset today.
`chp`	float	Change in price in % of security or asset today.
		cn - connection message sub - subscribe message

Attribute	Data Type	Description
`type`	string	if - index data message dp - market depth data message sf - Equity and option data message

Request samples

Python | Node | Web modular API | C# | Java

Copy

```python
from fyers_apiv3.FyersWebsocket import data_ws

def onmessage(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("Response:", message)


def onerror(message):
    """
    Callback function to handle WebSocket errors.

    Parameters:
        message (dict): The error message received from the WebSocket.


    """
    print("Error:", message)


def onclose(message):
    """
    Callback function to handle WebSocket connection close events.
    """
    print("Connection closed:", message)
```

```python
def onopen():
    """
    Callback function to subscribe to data type and symbols upon WebSocket co

    """
    # Specify the data type and symbols you want to subscribe to
    data_type = "SymbolUpdate"

    # Subscribe to the specified symbols and data type
    symbols = ["NSE:NIFTY50-INDEX" , "NSE:NIFTYBANK-INDEX"]
    fyers.subscribe(symbols=symbols, data_type=data_type)

    # Keep the socket running to receive real-time data
    fyers.keep_running()


# Replace the sample access token with your actual access token obtained from
access_token = "XC4XXXXXXM-100:eXXXXXXXXXXXXfZNSBoLo"

# Create a FyersDataSocket instance with the provided parameters
fyers = data_ws.FyersDataSocket(
    access_token=access_token,       # Access token in the format "appid:acce
    log_path="",                     # Path to save logs. Leave empty to auto
    litemode=False,                  # Lite mode disabled. Set to True if you
    write_to_file=False,              # Save response in a log file instead o
    reconnect=True,                  # Enable auto-reconnection to WebSocket
    on_connect=onopen,               # Callback function to subscribe to data
    on_close=onclose,                # Callback function to handle WebSocket
    on_error=onerror,                # Callback function to handle WebSocket
    on_message=onmessage             # Callback function to handle incoming m
)

# Establish a connection to the Fyers WebSocket
fyers.connect()
```

Sample Success Response

```
 {
    "symbol":"NSE:NIFTY50-INDEX",
     "ch":"-27.95",
     "high_price":"26277.35",
     "ltp":"26188.1",
     "exch_feed_time":"1727428424",
     "chp":"-0.11",
     "low_price":"26166.95",
     "open_price":"26248.25",
     "type":"if",
```

```
    "prev_close_price":"26216.05"
}
```


Market Data Depth Update

The WebSocket API for receiving stock data is a real-time communication protocol designed to provide seamless and low-latency access to live stock market data. This API allows developers and traders to establish a persistent, bidirectional connection with the server, enabling them to receive continuous updates on stock prices, trading volumes, and other relevant market information.

To quickly get started with the WebSocket API and start receiving real-time stock data, you can explore our sample scripts and code examples in our GitHub repository:
Data WebSocket Sample Scripts

Response Attributes(Depth Update)

Attribute	Data Type	Description
`symbol`	string	The symbol in fyers symbology format whose data is recieved on socket.
`bid_price1-bid_price5`	float	The bid price key represents the highest price at which buyers are willing to purchase a particular security or asset.
`ask_price1-ask_price5`	float	The ask price key represents the lowest price at which sellers are willing to sell a particular security or asset.
`bid_size1-bid_size5`	integer	The bid size key represents the quantity or volume of the security or asset that buyers are willing to purchase at a specific bid price level.
`ask_size1-ask_size5`	integer	The ask size key represents the quantity or volume of the security or asset that sellers are willing to sell at a specific ask price level.
`bid_order1-bid_order5`	integer	The number of bid order.
`ask_order1-ask_order5`	integer	The number of ask order.
`type`	string	cn - connection message sub - subscribe message if - index data message

Attribute	Data Type	Description
		dp - market depth data message sf - Equity and option data message

Request samples

Python | Node | Web modular API | C# | Java

Copy

```
from fyers_apiv3.FyersWebsocket import data_ws

def onmessage(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("Response:", message)


def onerror(message):
    """
    Callback function to handle WebSocket errors.

    Parameters:
        message (dict): The error message received from the WebSocket.


    """
    print("Error:", message)


def onclose(message):
    """
    Callback function to handle WebSocket connection close events.
    """
    print("Connection closed:", message)
```

```python
def onopen():
    """
    Callback function to subscribe to data type and symbols upon WebSocket co

    """
    # Specify the data type and symbols you want to subscribe to
    data_type = "DepthUpdate"

    # Subscribe to the specified symbols and data type
    symbols = ['NSE:SBIN-EQ', 'NSE:ADANIENT-EQ']
    fyers.subscribe(symbols=symbols, data_type=data_type)

    # Keep the socket running to receive real-time data
    fyers.keep_running()


# Replace the sample access token with your actual access token obtained from
access_token = "XXDHHIOKS-100:eajoljXXXXXXXXXX"

# Create a FyersDataSocket instance with the provided parameters
fyers = data_ws.FyersDataSocket(
    access_token=access_token,       # Access token in the format "appid:acce
    log_path="",                     # Path to save logs. Leave empty to auto
    litemode=False,                  # Lite mode disabled. Set to True if you
    write_to_file=False,              # Save response in a log file instead o
    reconnect=True,                  # Enable auto-reconnection to WebSocket
    on_connect=onopen,               # Callback function to subscribe to data
    on_close=onclose,                # Callback function to handle WebSocket
    on_error=onerror,                # Callback function to handle WebSocket
    on_message=onmessage             # Callback function to handle incoming m
)

# Establish a connection to the Fyers WebSocket
fyers.connect()
```

Sample Success Response

```
{
 "bid_price1":606.25,
 "bid_price2":606.2,
 "bid_price3":606.15,
 "bid_price4":606.1,
 "bid_price5":606.05,
 "ask_price1":606.3,
 "ask_price2":606.35,
```

```
  "ask_price3":606.4,
  "ask_price4":606.45,
  "ask_price5":606.5,
  "bid_size1":20,
  "bid_size2":902,
  "bid_size3":111,
  "bid_size4":110,
  "bid_size5":979,
  "ask_size1":282,
  "ask_size2":568,
  "ask_size3":2910,
  "ask_size4":1676,
  "ask_size5":2981,
  "bid_order1":1,
  "bid_order2":3,
  "bid_order3":2,
  "bid_order4":2,
  "bid_order5":9,
  "ask_order1":4,
  "ask_order2":2,
  "ask_order3":12,
  "ask_order4":9,
  "ask_order5":17,
  "type":"dp",
  "symbol":"NSE:SBIN-EQ"
}
```


Market Data Lite-Mode

The WebSocket API provides a lightweight and efficient "Lite Mode" for receiving only the Last Traded Price (LTP) updates of specific stock symbols. This mode is designed for users who require real-time access to LTP data without the additional overhead of subscribing to other stock data fields. The Lite Mode allows for seamless integration into applications where only the latest stock prices are needed.

Request samples

Python | Node | Web modular API | C# | Java

Copy

```
from fyers_apiv3.FyersWebsocket import data_ws


def onmessage(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("Response:", message)


def onerror(message):
    """
    Callback function to handle WebSocket errors.

    Parameters:
        message (dict): The error message received from the WebSocket.


    """
    print("Error:", message)


def onclose(message):
    """
    Callback function to handle WebSocket connection close events.
    """
    print("Connection closed:", message)


def onopen():
    """
    Callback function to subscribe to data type and symbols upon WebSocket co

    """
    # Specify the data type and symbols you want to subscribe to
    data_type = "SymbolUpdate"

    # Subscribe to the specified symbols and data type
    symbols = ['NSE:SBIN-EQ', 'NSE:ADANIENT-EQ']
    fyers.subscribe(symbols=symbols, data_type=data_type)

    # Keep the socket running to receive real-time data
    fyers.keep_running()
```

```
# Replace the sample access token with your actual access token obtained from
access_token = "XC4XXXXXXM-100:eXXXXXXXXXXXXfZNSBoLo"

# Create a FyersDataSocket instance with the provided parameters
fyers = data_ws.FyersDataSocket(
    access_token=access_token,       # Access token in the format "appid:acce
    log_path="",                     # Path to save logs. Leave empty to auto
    litemode=True,                  # Lite mode disabled. Set to True if you
    write_to_file=False,              # Save response in a log file instead o
    reconnect=True,                  # Enable auto-reconnection to WebSocket
    on_connect=onopen,               # Callback function to subscribe to data
    on_close=onclose,                # Callback function to handle WebSocket
    on_error=onerror,                # Callback function to handle WebSocket
    on_message=onmessage             # Callback function to handle incoming m
)

# Establish a connection to the Fyers WebSocket
fyers.connect()
```

```
--------------------------------------------------------------------------------
 Sample Success Response
--------------------------------------------------------------------------------

  {
    symbol: 'NSE:SBIN-EQ',
    ltp: 500.55,
    type: 'sf'
  }
```

Market Data Unsubscribe

To stop receiving real-time data updates for a specific stock symbol or a group of symbols, you can utilize the "unsubscribe" action in the WebSocket API. Sending an "unsubscribe" message will remove the specified symbol(s) from your active subscriptions, and you will no longer receive updates for those symbols. You can check the sample code to know how to unsubscribe to already subscribed symbols.

Request samples

Python | Node | Web modular API | C# | Java

Copy

```
from fyers_apiv3.FyersWebsocket import data_ws

def onmessage(message):
    """
    Callback function to handle incoming messages from the FyersDataSocket We

    Parameters:
        message (dict): The received message from the WebSocket.

    """
    print("Response:", message)
    # After processing or when you decide to unsubscribe for specific symbol
    # you can use the fyers.unsubscribe() method

    # Example of condition: Unsubscribe when a certain condition is met
    if message['symbol']== 'NSE:SBIN-EQ' and message['ltp'] > 610:
        # Unsubscribe from the specified symbols and data type
        data_type = "SymbolUpdate"
        symbols_to_unsubscribe = ['NSE:SBIN-EQ']
        fyers.unsubscribe(symbols=symbols_to_unsubscribe, data_type=data_type


def onerror(message):
    """
    Callback function to handle WebSocket errors.

    Parameters:
        message (dict): The error message received from the WebSocket.


    """
    print("Error:", message)


def onclose(message):
    """
    Callback function to handle WebSocket connection close events.
    """
    print("Connection closed:", message)


def onopen():
    """
```

```
    Callback function to subscribe to data type and symbols upon WebSocket co

    """
    # Specify the data type and symbols you want to subscribe to
    data_type = "SymbolUpdate"

    # Subscribe to the specified symbols and data type
    symbols = ['NSE:SBIN-EQ', 'NSE:ADANIENT-EQ']
    fyers.subscribe(symbols=symbols, data_type=data_type)

    # Keep the socket running to receive real-time data
    fyers.keep_running()


# Replace the sample access token with your actual access token obtained from
access_token = "XXDHHIOKS-100:eajoljXXXXXXXXXX"

# Create a FyersDataSocket instance with the provided parameters
fyers = data_ws.FyersDataSocket(
    access_token=access_token,       # Access token in the format "appid:acce
    log_path="",                     # Path to save logs. Leave empty to auto
    litemode=False,                  # Lite mode disabled. Set to True if you
    write_to_file=False,              # Save response in a log file instead o
    reconnect=True,                  # Enable auto-reconnection to WebSocket
    on_connect=onopen,               # Callback function to subscribe to data
    on_close=onclose,                # Callback function to handle WebSocket
    on_error=onerror,                # Callback function to handle WebSocket
    on_message=onmessage             # Callback function to handle incoming m
)

# Establish a connection to the Fyers WebSocket
fyers.connect()


------------------------------------------------------------------------------
 Sample Success Response
------------------------------------------------------------------------------

  {'type': 'unsub', 'code': 200, 'message': 'Unsubscribed', 's': 'ok'}
```

Order Websocket Usage Guide

Introduction

The WebSocket API for receiving order, position, trade, price alerts, and EDIS updates is a real-time communication protocol designed to provide seamless access to various critical elements of a trading and EDIS system. This API allows traders and developers to establish a persistent, bidirectional connection with the server, enabling them to receive real-time updates on their orders, current positions, executed trades, price alerts, and EDIS status.
This guide provides instructions on integrating the order WebSocket into any programming language.

Order WebSocket Connection

To connect to order websocket, the below two input params are mandate

1. Websocket endpoint : wss://socket.fyers.in/trade/v3
2. Header:
    - Format: **< appId:accessToken >**
    - Sample header format : 7ABXUX38S-100:eyJ0eXAi**********qiTnzd2lGwS17s

Based on the programming language chosen, respective socket connection libraries can be used.
For the reference please find the sample code for socket connection written in Python.

Here, we are making a connection with Fyers order websocket with parameters and callback functions on_message, on_error, on_close, on_open, which are required in the Python socket connection library used and would change as per the other programming library used. Sample callback function is shared below.

Note : Handle accordingly in your Programming language

Connection Response:

On Success: Returns the socket object

On Failure:

Possible Error : Status code 403

1. Error: Handshake status 403 Forbidden -+-+- {'date': 'Tue, 19 Dec 2023 04:46:45 GMT', 'content-

length': '0', 'connection': 'keep-alive', 'cf-cache-status': 'DYNAMIC', 'set-cookie': '__cf_bm=BOE16LGB7NHpNqW0AJOuFN1rcL3Q9TgnhmtpBfb3.Wk-1702961205-1-AfmECmK9cbVA2XGvkpx+jFuXyRsJET/ZOQYmw3LyZJ68pYLZTgtpbalvNs09ECZZ4GpPiogeYGhhFo+3 path=/; expires=Tue, 19-Dec-23 05:16:45 GMT; domain=.fyers.in; HttpOnly; Secure', 'server': 'cloudflare', 'cf-ray': '837d00eec8ed17b6-MAA'} -+-+- b"
Reason : This error will come when your accessToken is wrong
How to solve : Provide correct accessToken
AccessToken format: **< appID:accesstoken >**

2. Error: Handshake status 404 Not Found -+-+- {'date': 'Tue, 19 Dec 2023 10:04:35 GMT', 'content-type': 'text/plain; charset=utf-8', 'content-length': '0', 'connection': 'keep-alive', 'cf-cache-status': 'DYNAMIC', 'set-cookie': '__cf_bm=I5oN6zdeKjfGsicqFiXZ57J5SX2IjsFDspaLIEGKPDE-1702980275-1-Ae+ldjrb3WfSuM7yNOzo3ykOIBQ1m+50QqcIqU26A+wqPIHhIEIGSy9kT2OG3OWNI0hwcmh7U+PnJ/aW path=/; expires=Tue, 19-Dec-23 10:34:35 GMT; domain=.fyers.in; HttpOnly; Secure', 'server': 'cloudflare', 'cf-ray': '837ed28169c817ae-MAA'} -+-+- b"
Reason : Socket Connection URL would be wrong
How to solve : Provide valid URL.

Sample callback function:

For more reference, please find our on_open callback function code for more reference

Request samples

Python

Copy

```
try:
if self.__ws_object is None:
    if self.write_to_file:
        self.background_flag = True
    header = {"authorization": self.__access_token}
    ws = websocket.WebSocketApp(
        self.__url,
        header=header,
        on_message=lambda ws, msg: self.__on_message(msg),
        on_error=lambda ws, msg: self.On_error(msg),
        on_close=lambda ws, close_code, close_reason: self.__on_close(
            ws, close_code, close_reason
        ),
        on_open=lambda ws: self.__on_open(ws),
    )
    self.t = Thread(target=ws.run_forever)
```

```
        self.t.daemon = self.background_flag
        self.t.start()

    #Sample callback function:

        def __on_open(self, ws):
        try:
            if self.__ws_object is None:
            self.__ws_object = ws
            self.ping_thread = threading.Thread(target=self.__ping)
            self.ping_thread.start()
        except Exception as e:
            self.order_logger.error(e)
            self.On_error(e)
```

Subscribe Method

Once the connection is established, it is required to subscribe for required actions to get the data. To subscribe for different actions, create a message data, which would be the string format for json node.

```
message = json.dumps( {"T": "SUB_ORD", "SLIST": action_data, "SUB_T": 1} )
```

Json node Params:

T: Type: String
value: “SUB_ORD” (Fixed)

action_data: Type: List/Array
Value: ['orders', 'trades', 'positions', 'edis', 'pricealerts', 'login'] Note: Based on the list passed in action_data web_socket data will be received

SUB_T: Integer
Value: 1 (value 1 is for subscribing and -1 for unsubscribe)

Convert the json to string and send this message to socket to subscribe for action_data mentioned

Sample Response:

```
{'code': 1605, 'message': 'Successfully subscribed', 's': 'ok'}
```

Response from socket on any action triggered

Once the subscribed successfully, for any action triggered, data will be received through socket, if any

callback function is defined, would receive on function.

Response would be string, In node we get as array buffer and in python we string, then it parsed to required format. In another programming language you might get in another format you just have to change in string.

Type: string
Value: {"orders":
{"client_id":"XP0001","exchange":10,"fy_token":"10100000001628","id":"23121800292158","id_fyers":"df013f5
6925-4e2d-ba0f-
0becf1229298","instrument":0,"lot_size":1,"offline_flag":false,"oms_flag":"K:1","ord_source":"W","ord_status":2
Ack","segment":10,"status_msg":"New Ack","symbol":"NSE:BECTORFOOD-EQ","symbol_desc":"MRS
BECTORS FOOD SPE LTD","symbol_exch":"BECTORFOOD",
"tick_size":0.05,"time_epoch_oms":1702887690,"time_exch":"NA","time_oms":"18-Dec-2023
13:51:30","tran_side":1,"update_time_epoch_oms":1702887690,"update_time_exch":"01-Jan-1970
05:30:00","validity":"DAY"},"s":"ok"}

Note:

1. Above response would be in the string format (In Python) and arraybuffer (In Node). You have to check in which format you are getting data from websocket as it is dependent on the websocket library for the particular language.
2. Once you get data, you have to change into the string.(if already in string format then no need to change). After that, you have to change this string to JSON. Here you find the following keys as a JSON Key (One of them) : orders, trades, positions.
3. Now Based on the Key the data is identified, if key is orders it means it is order update message, if key is trades then this message is for trades updates and same for positions updates.
4. In an Fyers SDK the socket raw response is parsed to generate data with required keys and remove the unnecessary keys

   Parsed Data: {'s': 'ok', 'orders': {'clientId': 'XP03754', 'id': '23121800388066', 'qty': 1, 'remainingQuantity': 1, 'type': 2, 'fyToken': '10100000002705', 'exchange': 10, 'segment': 10, 'symbol': 'NSE:PRAJIND-EQ', 'instrument': 0, 'offlineOrder': False, 'orderDateTime': '18-Dec-2023 16:33:24', 'orderValidity': 'DAY', 'productType': 'CNC', 'side': 1, 'status': 4, 'source': 'W', 'ex_sym': 'PRAJIND', 'description': 'PRAJ INDUSTRIES LTD', 'orderNumStatus': '23121800388066:4'}}

   All the keys information for orders updates are available there : Link
   All the keys information for Trades updates are available there : Link
   All the keys information for positions updates are available there : Link

5. Also attaching our internal mapping for your reference, how we are changing the keys from raw data to final data.

```
"position_mapper" :
 {
        "symbol": "symbol",
        "id": "id",
        "buy_avg": "buyAvg",
        "buy_qty": "buyQty",
        "buy_val": "buyVal",
        "sell_avg": "sellAvg",
        "sell_qty": "sellQty",
```

```
                "sell_val": "sellVal",
                "net_avg": "netAvg",
                "net_qty": "netQty",
                "tran_side": "side",
                "qty": "qty",
                "product_type": "productType",
                "pl_realized": "realized_profit",
                "rbirefrate": "rbiRefRate",
                "fy_token": "fyToken",
                "exchange": "exchange",
                "segment": "segment",
                "day_buy_qty": "dayBuyQty",
                "day_sell_qty": "daySellQty",
                "cf_buy_qty": "cfBuyQty",
                "cf_sell_qty": "cfSellQty",
                "qty_multiplier": "qtyMulti_com",
                "pl_total": "pl",
                "cross_curr_flag": "crossCurrency",
                "pl_unrealized": "unrealized_profit"
        },
          "order_mapper" :
        {
                "client_id":"clientId",
                "id":"id",
                "id_parent":"parentId",
                "id_exchange":"exchOrdId",
                "qty":"qty",
                "qty_remaining":"remainingQuantity",
                "qty_filled":"filledQty",
                "price_limit":"limitPrice",
                "price_stop":"stopPrice",
                "tradedPrice":"price_traded",
                "ord_type":"type",
                "fy_token":"fyToken",
                "exchange":"exchange",
                "segment":"segment",
                "symbol":"symbol",
                "instrument":"instrument",
                "oms_msg":"message",
                "offline_flag":"offlineOrder",
                "time_oms":"orderDateTime",
                "validity":"orderValidity",
                "product_type":"productType",
                "tran_side":"side",
                "org_ord_status":"status",
                "ord_source":"source",
                "symbol_exch":"ex_sym",
                "symbol_desc":"description"
          },
            "trade_mapper" :
          {
                "id_fill": "tradeNumber",
                "id": "orderNumber",
                "qty_traded": "tradedQty",
                "price_traded": "tradePrice",
                "traded_val": "tradeValue",
                "product_type": "productType",
                "client_id": "clientId",
                "id_exchange": "exchangeOrderNo",
                "ord_type": "orderType",
                "tran_side": "side",
                "symbol": "symbol",
                "fill_time": "orderDateTime",
                "fy_token": "fyToken",
                "exchange": "exchange",
                "segment": "segment"
          }
```

Request samples

Python

Copy

```
def subscribe(self, data_type: str) -> None:
"""
Subscribes to real-time updates of a specific data type.

Args:
    data_type (str): The type of data to subscribe to, such as orders, positi


"""

try:
    if self.__ws_object is not None:
        self.data_type = []
        for elem in data_type.split(","):
            if isinstance(self.socket_type[elem], list):
                self.data_type.extend(self.socket_type[elem])
            else:
                self.data_type.append(self.socket_type[elem])

        print("Data type is ", self.data_type)
        print("Data type is ", type(self.data_type))
        message = json.dumps(
            {"T": "SUB_ORD", "SLIST": self.data_type, "SUB_T": 1}
        )
        self.__ws_object.send(message)

except Exception as e:
    self.order_logger.error(e)
```

UnSubscribe Method

To Unsubscribe for different actions, create a message data, which would be the string format for json node.

```
message = json.dumps( {"T": "SUB_ORD", "SLIST": action_data, "SUB_T": -1} )
```

Json node Params:

T: Type: String
value: “SUB_ORD” (Fixed)

action_data: Type: List/Array
Value: ['orders', 'trades', 'positions', 'edis', 'pricealerts', 'login'] Note: Based on the list passed in action_data web_socket data will be received

SUB_T: Integer
Value: -1 (value -1 is for unsubscribing and 1 for subscribe)

Convert the json to string and send this message to socket to subscribe for action_data mentioned

Request samples

Python

Copy

```
def unsubscribe(self, data_type: str) -> None:
  """
    Unsubscribes from real-time updates of a specific data type.

  Args:
  data_type (str): The type of data to unsubscribe from, such as orders, posi

  """

  try:
    if self.__ws_object is not None:
      self.data_type = [
        self.socket_type[(type)] for type in data_type.split(",")
      ]
      message = json.dumps(
        {"T": "SUB_ORD", "SLIST": self.data_type, "SUB_T": -1}
      )
      self.__ws_object.send(message)

except Exception as e:
  self.order_logger.error(e)
```

Ping Method

To check whether the websocket connection is alive or not, we have to send a periodically “ping” message. If we get a pong from websocket, it means it is alive else dead. Find how we are doing in Python Code.

Here there is one while loop with sleep of 10 seconds and we send a “ping” message to websocket to know that websocket is alive or not.

Request samples

Python

Copy

```python
def __ping(self) -> None:
  """
  Sends periodic ping messages to the server to maintain the WebSocket connec

  The method continuously sends "__ping" messages to the server at a regular
  as long as the WebSocket connection is active.

  """

  while (
      self.__ws_object is not None
      and self.__ws_object.sock
      and self.__ws_object.sock.connected
  ):
      self.__ws_object.send("ping")
      time.sleep(10)
```

Mandatory fields

Method	Fields	Reason
WebSocket Connection	WebSocket Connection URL(Endpoint)	To make a connection
WebSocket Connection	Headers	To authorize
Subscribe Method	action_data	To subscribe for particular actions
Subscribe Method	"SUB_T": 1	While subscribing to any action
UnSubscribe Method	"SUB_T": -1	While unsubscribing any action

Tick-by-Tick (TBT) Websocket Usage Guide

Introduction

Tick-by-tick data is the most detailed market data, recording every trade and order book update in real-time. Each "tick" includes the price, volume, and timestamp of individual trades, as well as changes to buy and sell orders. This granular data is crucial for analyzing market microstructure, tracking order flow, and developing high-frequency trading strategies.

Key Points:

1. **Available for NFO Instruments Only**
   - Tick-by-tick data is exclusively available only for **NFO (NSE Futures & Options) instruments.** NSECM segment will be released soon.
2. **Data Formats**
   - **Requests** are sent in **JSON** format.
   - **Responses** are received in **protobuf** format (a compact, efficient data format).
3. **Incremental Data Updates**
   - Instead of sending the full market data repeatedly, the server only sends **changes (differences)** between the last data packet and the current one.

- To get the complete picture, users must maintain previous data and **apply these changes.**
- The official SDKs provided by **Fyers** will handle this process automatically.

4. **Snapshot Data on New Subscriptions**
   - When a user **subscribes to tick-by-tick data**, the first packet received is a **snapshot**, containing the full market data at that moment.
   - After this, all subsequent packets only contain **updates (differences)** that need to be applied to the snapshot for a real-time view.

TBT WebSocket Connection [50 Market Depth]

Currently, these are the features available on the socket

Feature	Description	Status
TBT 50 Market Depth	TBT 50 Market Depth provides users wtih 50 levels of market depth. Learn more	Available
TBT Quotes	TBT Quotes provides users with quote data such as ltp, ltt, ltq, oi etc	Coming soon
TBT DayQuote	TBT DayQuote provides users with day quote data such as day ohlcv oi	Coming soon
TBT OHLCV	TBT OHLCV provides users with ohlcv data on the smallest timeframe. This is the most accurate data as spikes are also captured in this data.	Coming soon

To connect to tbt websocket, the below input params are mandated

1. Websocket endpoint : wss://rtsocket-api.fyers.in/versova
2. Header:
   Key: Authorization
   - Format: **< appId:accessToken >**
   - Sample header format : 7ABXUX38S-100:eyJ0eXAi**********qiTnzd2lGwS17s

Concept of channels

With the Tick-by-Tick (TBT) WebSocket, we are introducing the concept of channels. A channel acts as a logical grouping for different subscribed symbols, making it easier to manage data streams efficiently.

How Channels Work

- When subscribing to market data, you need to specify both the symbols and a channel number.
- Simply subscribing to a channel does not start the data stream—you must also resume the channel to begin receiving updates.

Example Usage

Let’s say you organize your subscriptions as follows:

- Channel 1: All Nifty-related symbols
- Channel 2: All BankNifty-related symbols

Now, depending on what data you need, you can control the channels dynamically:

- To receive only Nifty data → Pause Channel 2 and Resume Channel 1
- To receive only BankNifty data → Pause Channel 1 and Resume Channel 2
- To receive both Nifty and BankNifty data → Resume both channels

This approach provides greater flexibility and control over market data streaming, allowing you to filter and manage real-time data efficiently.

Request Message Types

Type	Purpose	Encoding Format	Data Format
Ping	Ping message to keep connection alive	String	`"ping"`
			`{ "type": 1,`

Subscribe	Subscribe to the symbols for which data is required	Json string	<pre>"data": {
			"subs": 1,
			"symbols": ["NSE:IOC25FEBFUT",
			"NSE:BANKNIFTY25FEBFUT", "NSE:
			"mode": "depth",
			"channel": "1"
			}
			}</pre>

Unsubscribe	Unsubscribe to the symbols for which data is NOT required	Json string	<pre>{
			"type": 1,
			"data": {
			"subs": -1,
			"symbols": ["NSE:IOC25FEBFUT",
			"NSE:BANKNIFTY25FEBFUT", "NSE:
			"mode": "depth",
			"channel": "1"
			}
			}</pre>

Switch Channel	Switch between active and paused channels	Json string	<pre>{
			"type": 2,
			"data": {
			"resumeChannels": ["1"],
			"pauseChannels": []
			}
			}</pre>

Response Message Types

We use Protocol Buffers (protobuf) as the response format for all market data. The .proto file, which defines the data structure, is available at:

📌 Proto URL: https://public.fyers.in/tbtproto/1.0.0/msg.proto

Protobuf is a widely used data format, and compilers are available to generate code in different programming languages.

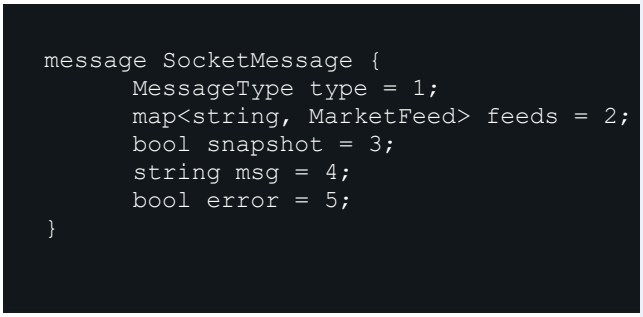
How to Install and Use Protobuf

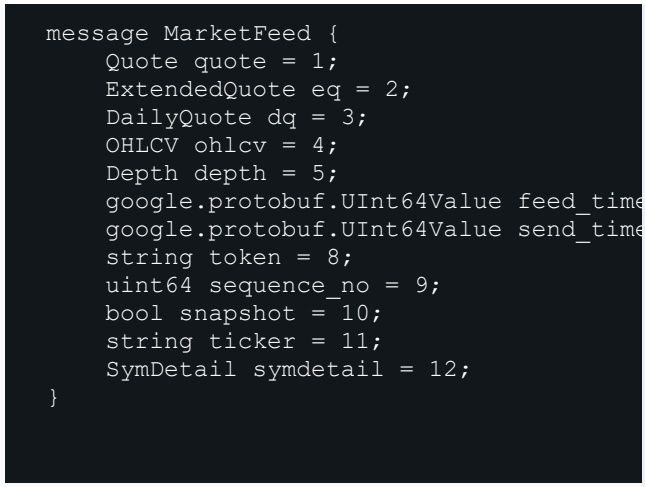
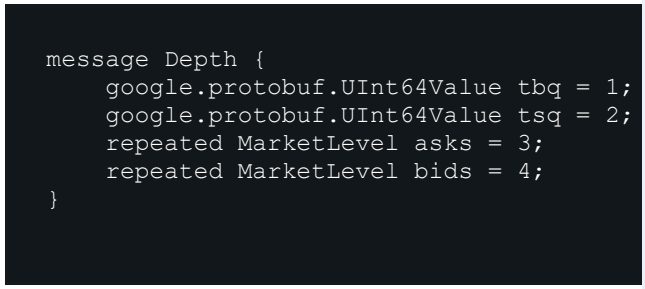
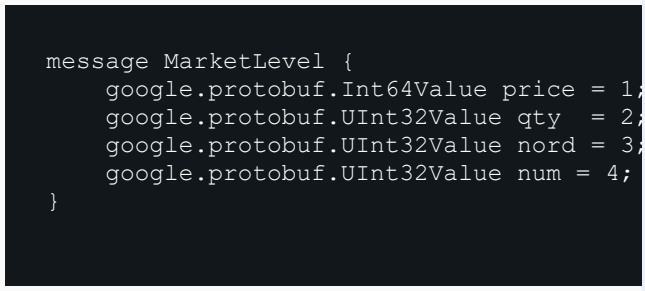
- Protobuf Compiler Installation: https://protobuf.dev/getting-started/
- Using Protobuf with Python: https://protobuf.dev/reference/python/python-generated/
- Using Protobuf with Node.js: https://www.npmjs.com/package/protobufjs

We have uploaded the compiled files also for python, nodejs, and golang. You can download the files from the below link: https://public.fyers.in/tbtproto/1.0.0/protogencode.zip
Copy the required file directly in your project and use it.

Proto Versions and Links:

Proto Version	Proto URL	Compiled files URL
1.0.0	https://public.fyers.in/tbtproto/1.0.0/msg.proto	https://public.fyers.in/tbtproto/1.0.0/prot

Type	Data Format	Field Explanation
SocketMessage	• type: value will always be MessageType.depth • feeds: string (key) will be the symbol ticker and value will be the MarketFeed datastructure. This value will have the actual 50 depth values • snapshot: true if its a snapshot and false if its a diff packet • msg: will mostly contain error msgs when error = true • error: true if it is an error, else false. In case of true, feeds will be nil/empty	
	• depth: Depth datastructure. This field will have the actual market depth value	

MarketFeed	• feed_time: time of the packet in unix epoch • send_time: time of the packet at the time of sending from server • token: fytoken for the symbol • snapshot: true if its a snapshot else false for a diff packet • ticker: symbol ticker **Note:** other fields can be ignored	
Depth	• tbq: total bid qty • tsq: total sell qty • asks: array of asks of type Marketlevel datastructure • bids: array of bids of type Marketlevel datastructure	
MarketLevel	• price: price • qty: qty in the market depth for the price • nord: number of orders in the market depth for the price • num: depth number, for 50 depth will be between 0 and 49 [0 based array indexing]	

Ratelimits

Following ratelimits apply for TBT Websocket:

Description	Limit
Active Connections Per App Per User	3
Symbols per connection [Market Depth]	5
Channels per connection	50 (1-50)

True Data V2

Introduction

Fyers is offering market data APIs in collaboration with Truedata. The data APIs provide both real-time data as well as historical data.

Truedata API Documentation

You can download the entire Truedata API document **here.** This document will provide all the different functions which are available, sample requests and responses along with the Python sample code. You can also visit the official Python package on PyPi **here**

Prerequisites

Prerequisites to use Python Truedata Websocket library

1. Python 3.7 or above
2. pip install truedata_ws

How to subscribe to Truedata from FYERS ?

You can subscribe to the different Truedata subscription plans from **here** You can view your subscription status from **here**

Appendix

Fytoken

Indicator	Format	Description
Exchange	2 Digits	Exchange of the symbol
Segment	2 Digits	Segment of the symbol
Expiry	6 Digits	Format: YYMMDD Eg: 200827
Exchange Token`	From 2 upto 6 Digits	The token assigned for the symbol by the exchange Eg:22

Exchanges

Possible Values	Description
10	NSE (National Stock Exchange)
11	MCX (Multi Commodity Exchange)
12	BSE (Bombay Stock Exchange)

Segments

Possible Values	Description
10	Capital Market
11	Equity Derivatives
12	Currency Derivatives
20	Commodity Derivatives

Available Exchange-Segment Combinations

Exchange	Segment	Exchange Code	Segment Code
NSE	Capital Market	10	10
NSE	Equity Derivatives	10	11
NSE	Currency Derivatives	10	12
NSE	Commodity Derivatives	10	20
BSE	Capital Market	12	10
BSE	Equity Derivatives	12	11
BSE	Currency Derivatives	12	12
MCX	Commodity Derivatives	11	20

Instrument Types

Possible Values	Description
.	**CM segment**
0	EQ (EQUITY)
1	PREFSHARES
2	DEBENTURES
3	WARRANTS
4	MISC (NSE, BSE)
10	INDEX
50	MISC (BSE)
.	**FO segment**
11	FUTIDX
12	FUTIVX
13	FUTSTK
14	OPTIDX
15	OPTSTK
.	**CD segment**
16	FUTCUR
17	FUTIRT
18	FUTIRC
19	OPTCUR
20	UNDCUR
21	UNDIRC
22	UNDIRT
23	UNDIRD
24	INDEX_CD
25	FUTIRD

Possible Values	Description
.	**COM segment**
11	FUTIDX
30	FUTCOM
31	OPTFUT
32	OPTCOM

Symbology Format

Segment	Format	Examples
Equity	{Ex}:{Ex_Symbol}-{Series}	NSE:SBIN-EQ, NSE:ACC-EQ, NSE:MODIRUBBER-BE BSE:SBIN-A, BSE:ACC-A, BSE:MODIRUBBER-T
Equity Futures	{Ex}:{Ex_UnderlyingSymbol}{YY}{MMM}FUT	NSE:NIFTY20OCTFUT, NSE:BANKNIFTY20NOVFUT, BSE:SENSEX23AUGFUT
Equity Options (Monthly Expiry)	{Ex}:{Ex_UnderlyingSymbol}{YY}{MMM}{Strike}{Opt_Type}	NSE:NIFTY20OCT11000CE, NSE:BANKNIFTY20NOV25000PE , BSE:SENSEX23AUG60400CE
Equity Options (Weekly Expiry)	{Ex}:{Ex_UnderlyingSymbol}{YY}{M}{dd}{Strike}{Opt_Type} refer here for possible values of {M}	NSE:NIFTY2010811000CE, NSE:NIFTY20O0811000CE, BSE:SENSEX2381161000CE, NSE:NIFTY20D1025000CE
Currency Futures	{Ex}:{Ex_CurrencyPair}{YY}{MMM}FUT	NSE:USDINR20OCTFUT, NSE:GBPINR20NOVFUT
Currency Options (Monthly Expiry)	Ex}:{Ex_CurrencyPair}{YY}{MMM}{Strike}{Opt_Type}	NSE:USDINR20OCT75CE, NSE:GBPINR20NOV80.5PE
Currency Options (Weekly Expiry)	{Ex}:{Ex_CurrencyPair}{YY}{M}{dd}{Strike}{Opt_Type}	NSE:USDINR20O0875CE, NSE:GBPINR20N0580.5PE NSE:USDINR20D1075CE
Commodity Futures	{Ex}:{Ex_Commodity}{YY}{MMM}FUT	MCX:CRUDEOIL20OCTFUT, MCX:GOLD20DECFUT

Segment	Format	Examples
Commodity Options (Monthly Expiry)	{Ex}:{Ex_Commodity}{YY}{MMM}{Strike}{Opt_Type}	MCX:CRUDEOIL20OCT4000CE, MCX:GOLD20DEC40000PE

Symbology Possible Values

Variable	Explanation	Possible Values
{Ex}	This is the exchange on which the symbol is trading	NSE, BSE, MCX
{YY}	This is the last 2 digits of the year	2019 will be 19 2020 will be 2020 2021 will be 21 2022 will be 22
{MMM}	For monthly expiries such as futures and options, this value is the 3 characters of the month. The month will always be denoted in uppercase.	JAN FEB MAR APR MAY JUN JUL AUG SEP OCT NOV DEC
{M}	For weekly expiries, this value represents the month of expiry but with only 1 character. This is a mix of numbers as well as characters.	Jan => 1 Feb => 2 Mar => 3 Apr => 4 May => 5 Jun => 6 Jul => 7 Aug => 8 Sep => 9 Oct => O (Letter) Nov => N Dec => D
		01

Variable	Explanation	Possible Values
{dd}	This is used in weekly expiries. This represents the date of the month on which the contract is going to expire. This value will always be 2 characters.	06 10 25 30
{Opt_Type}	This represents whether the contract is call or put	Call Option => CE, Put Option => PE
{Strike}	This is the strike price of the option contract	11000 75.5

Product Types.

Possible Values	Description
CNC	For equity only
INTRADAY	Applicable for all segments
MARGIN	Applicable only for derivatives
CO	Cover Order
BO	Bracket Order
MTF	Margin Trading Facility

Order Types.

Possible Values	Description
1	Limit order
2	Market order
3	Stop order (SL-M)
4	Stoplimit order (SL-L)

Order Status

Possible Values	Description
1	Cancelled
2	Traded / Filled
3	For future use
4	Transit
5	Rejected
6	Pending

Order Sides

Possible Values	Description
1	Buy
-1	Sell

Position Sides

Possible Values	Description
1	Long
-1	Short
0	Closed position

Holding Types

Possible Values	Description
T1	The shares are purchased but not yet delivered to the demat account.
HLD	The shares are purchased and are available in the demat account.

Order Sources

Possible Values	Description
M	Mobile
W	Web
R	Fyers One
A	Admin
ITS	API

Change Log

Date	Changes
`20 Feb 2025`	• **TBT 50 depth** websocket documentation
`13 Feb 2025`	• Added keys for DDPI and MTF Activation Status in Profile API
	• Introduced Logout API **click here**

Date	Changes

`05 Feb 2025`	**Version**:
	`Node : fyers-api-v3 - v1.4.0`
	`Web Js : fyers-web-sdk-v3 - v1.3.0`
	`CDN :fyers-web-sdk-v3 - v1.3.0`
	`Java : fyers-apiv3 - v1.1.0`
	• Added MTF and GTT Order Placement

`01 Feb 2025`	**Version**: `C# : fyers-apiv3 - v1.0.6` • Added MTF and GTT Order Placement
`30 Jan 2025`	**Version**: `Python : fyers-apiv3 - v3.1.5` • Added MTF and GTT Order Placement
`20 Jan 2025`	• Introduced MTF and GTT Order Placement APIs

`13 Jan 2025`	**Version**:
	`C# : fyers-api-v3 - v1.0.5`
	`Java : fyers-apiv3 - v1.0.2`
	• Added key for DisclosedQty in modify order

`27 Dec 2024`	**Version**:
	`Node : fyers-api-v3 - v1.3.3`
	`Web Js : fyers-web-sdk-v3 - v1.2.4`
	`CDN :fyers-web-sdk-v3 - v1.2.1`
	• Improved compatibility with `symbol` s containing special characters

`26 Dec 2024`	**Version**:
	`C# : fyers-api-v3 - v1.0.4`
	`Java : fyers-apiv3 - v1.0.1`
	• Fixed Data Socket Issue for Sensex `Symbol`

`10 Oct 2024`	**Version**: `Python : fyers-apiv3 - v3.1.4` • Minor bug fixes within websocket package
`09 Oct 2024`	**Version**: `C# : fyers-api-v3 - v1.0.3` • Added Multileg order feature

Date	Changes
`03 Oct 2024`	**Version**: `Java : fyers-api-v3 - v1.0.0` • Introduced API V3 Java SDK
`17 Sep 2024`	**Version**: `C# : fyers-api-v3 - v1.0.2` • Added Option Chain API
`22 Aug 2024`	**Version**: `All` • Increased API rate limit from 10,000 requests per day to 1 Lakh requests per day (10x increase from the previous daily rate limit), **click here** to view the current rate limits. **Note:** There are no change in per-second and per-minute rate limits • Updated API document with API Error Codes **refer here**

`03 Jul 2024`	**Version**:
	`Python : fyers-apiv3 - v3.1.2`
	`Node : fyers-api-v3 - v1.3.0`
	• Added Multileg order feature

`12 Jun 2024`	**Version**: `C# : fyers-api-v3 - v1.0.1` • Optimized package with bug fixes
`07 Jun 2024`	**Version**: `Web Js : fyers-web-sdk-v3 - v1.2.2` • Optimized package
`30 May 2024`	**Version**: `All • Added description for fyers 201 code(order placement)
`29 May 2024`	**Version**: `Python : fyers-apiv3 - v3.1.0` • Improvised logging
`14 May 2024`	**Version**: `Web Js : fyers-web-sdk-v3 - v1.2.1` • Improvised in internal package import logic
	Version:

Date	Changes
`25 Apr 2024`	`C# : fyers-api-v3 - v1.0.0` • Introduced API V3 C# SDK

`23 Apr 2024`	**Version**:
	`Python : fyers-apiv3 - v3.1.0`
	`Node : fyers-api-v3 - v1.2.0`
	`Web Js : fyers-web-sdk-v3 - v1.2.0`
	`CDN :fyers-web-sdk-v3 - v1.2.0`
	• Added option chain API

`22 Mar 2024`	**Version**: `All` • Added 2 new reserved columns in `Symbol` Master CSV files, will be used for future development, kindly ignore
`04 Apr 2024`	**Version**: `All` • Deprecated **cmd** attribute in Quotes API response **click here** for more details

`26 Mar 2024`	**Version**:
	`Python : fyers-apiv3 - v3.1.0`
	`Node : fyers-api-v3 - v1.1.1`
	`Web Js : fyers-web-sdk-v3 - v1.1.0`
	`CDN :fyers-web-sdk-v3 - v1.1.0`
	• **`Symbol` Subscription Enhancement**
	◦ Optimised `symbol` subscription
	◦ Increased `symbol` subscription limit from **200** to **5000** in a single connection

`22 Mar 2024`	**Version**: `All` • Added 2 new reserved columns in `Symbol` Master CSV files, will be used for future development, kindly ignore
`15 Mar 2024`	**Version**: `fyers-web-sdk-v3 - v1.0.0` • Introduced Browser Compatible SDK.
	Version: `Python SDK - v3.0.9` • **Logging Enhancement**

Date	Changes
`14 Mar 2024`	- In the FyersModel class, the log_level parameter has been introduced to control the verbosity of logging. By default, the logging level is set to 'ERROR',only errors will be logged. However, users have the flexibility to set the log_level parameter to 'DEBUG', allowing for more detailed logging. Logs are written **fyersApi.log** file. - Added **fyersRequest.log** file where requested endpoints and status code are logged (can be used to analyse the API requests made). **Compatible with Python v3.12** - Updated aiohttp to version 3.9.3 (latest release) to ensure compatibility with Python 3.12.
`13 Mar 2024`	**Version**: `All` Added support for seconds candles
`05 Dec 2023`	Version: `All` **Order Placement** - OrderTagging functionality added. **Orderbook** - Orderbook returns Ordertag in response - Query orderbook by Ordertag **Tradebook** - Orderbook returns Ordertag in response - Query tradebook by Ordertag
	Version: `API Version 3` **New Features and Updates** **Performance** - Fast Order Placement: Achieving placement within < 75 ms - Fast get APIs: Achieving response times within < 75 ms **New Data Socket:** - Improved tick update speed, ensuring swift and efficient market data updates. - Introducing Lite mode for targeted last traded price (LTP) change updates. `Symbol` Update: Real-time `symbol` -specific updates for instant parameter changes. - DepthUpdate: Real-time market depth changes for selected `symbol` s. - Increased subscription capacity, accommodating tracking of 200 `symbol` s. - Strengthened error handling callbacks for seamless issue resolution. **New Order Socket**

Date	Changes
`18 Aug 2023`	◦ Real-time updates for orders ◦ Real-time updates for positions ◦ Real-time updates for trades ◦ Real-time updates for eDIS ◦ Real-time updates for price alerts ◦ Improved error handling callbacks • **URL update for various APIs** ◦ Enhanced History API available at: **https://api-t1.fyers.in/data/history** ◦ New Market Status API introduced at: **https://api-t1.fyers.in/data/marketStatus** ◦ Enhanced Quotes API available at: **https://api-t1.fyers.in/data/quotes** ◦ Enhanced Market Depth API available at: **https://api-t1.fyers.in/data/depth** • **Documentation and Sample Code Updates** ◦ Added sample code for NodeJS and Python in the documentation ◦ Updated sample code in the GitHub repository for both Python and NodeJS
`04 May 2023`	**Version**: `All` • **Quotes** ◦ URL updated to **https://api-t1.fyers.in/data/quotes?`symbol`s=NSE:SBIN-EQ** ◦ ip updated to lp ◦ Removed fyToken & prev_close_price from response ◦ Text updation: "`Symbol`s" changed to "`symbol`" • **Market Depth** ◦ URL updated to **https://api-t1.fyers.in/data/depth?`symbol`=NSE:SBIN-EQ&ohlcv_flag=1** ◦ itt updated to ltt ◦ Removed tick_Size & oi from response
	Version: `API Version 2` • **App Related** ◦ Introduced common apps for third party app developers ◦ Different authentication methods such as OAuth2, PKCE and OIDC ◦ Permission templates at the time of app creation • **Authentication** ◦ Authentication flow has been changed to improve security ◦ Authorization header value has been changed • **Order placement**

Date	Changes
-	◦ Multi order placement ◦ Multi order modification ◦ Multi order cancellation • **Other** ◦ Market status API **Realtime Data Rest API** ◦ Quotes api to get realtime data for list of `symbol`s via Rest API ◦ Market Depth API to get bid ask data for a particular `symbol` **Websocket** ◦ Get realtime price info for a list of `symbol`s ◦ Get realtime market depth for a particular `symbol` ◦ Get realtime order updates ◦ Background and Foreground process enabled for websocket. ◦ User can subscribe to only Index `symbol`s ◦ Added sample scripts to get started with websocket ◦ Object initialization method changed a bit (instead of fyers_api.websocket it is fyers_api.Websocket) ◦ No More Pong received message on the console. The data will be continuous **Postback (Webhooks)** ◦ Get realtime order updates via postbacks / webhooks **Transactions** ◦ eDIS flow